M02 — Prácticas integrales: programar y probar APIs REST con .NET 8

Autor: JAIME GALLO

Metodología de trabajo

Cada apartado parte del diseño REST explicado en teoría y culmina en **código ejecutable y peticiones HTTP**. Se trabaja con Visual Studio, ASP.NET Core 8 Minimal API, EF Core 8 y SQL Server Express LocalDB. Los cuatro estados de referencia están en `PROYECTO/2.1` a `PROYECTO/2.4`: no son simulaciones, sino proyectos con endpoints reales probados por HTTP.

La actividad práctica no requiere operaciones Git. El alumnado construye primero la etapa 2.1, incorpora los verbos HTTP en 2.2, conecta la validación en 2.3 y documenta el mismo servicio en 2.4. Para contrastar el resultado, puede abrir por separado los proyectos de referencia. La copia no sustituye la escritura y comprensión de código.

Persistencia: la API del alumno utiliza SQL Server Express LocalDB. Los tests de integración usan EF Core InMemory únicamente para ejecutarse de forma independiente en GitHub Actions. `EnsureCreated` se emplea en desarrollo para simplificar el arranque del ejercicio; no se propone para despliegues productivos.

Cómo construir las cuatro etapas sin duplicar código

Cada punto tiene su **proyecto de referencia terminado** en `PROYECTO/2.1`, `PROYECTO/2.2`, `PROYECTO/2.3` o `PROYECTO/2.4`. La referencia es la evidencia del estado final, no un archivo que deba copiarse antes de empezar. El alumno crea su propia solución, añade cada archivo y modifica lo que ya ha escrito al avanzar de paso. **No se deben pegar dos versiones completas de una misma clase o de un mismo endpoint.**

Los fragmentos se presentan según la etapa en la que el alumno debe introducirlos. El recorrido es **diseño → archivo → código → compilación → petición o prueba → resultado esperado**. Cuando un paso solo examina un comportamiento incorporado anteriormente, se indica expresamente y se evita volver a crear el código. Los retos son ampliaciones opcionales, no requisitos para que los proyectos de referencia compilen.

Criterio de finalización: al completar cada punto, el código de los archivos indicados debe equivaler funcionalmente al proyecto correspondiente y superar sus pruebas. La aplicación local usa SQL Server Express LocalDB; solo los tests sustituyen ese proveedor por InMemory.

Preparación común del puesto de trabajo

Herramientas: Visual Studio con desarrollo ASP.NET y web, SDK .NET 8 y, para ejecutar la API local, SQL Server Express **LocalDB** en Windows. Comprueba en una terminal `dotnet --list-sdks` y `sqllocaldb info MSSQLLocalDB`; no confundas la compilación con la disponibilidad de la base. En Visual Studio, `Ctrl+S` guarda, **Compilar solución** comprueba el código y **F5** inicia el perfil seleccionado. La dirección real de Kestrel se obtiene en la salida `Now listening on:`.

Carpetas de trabajo: crea una carpeta propia para cada etapa, sin editar el código de referencia. Para 2.1, el resultado esperado al terminar incluye `src/AceriaApp.Api`, `tests/AceriaApp.Api.Tests` y `AceriaApp.http` bajo la misma raíz del punto. En el asistente de Visual Studio, elige como ubicación de la API la carpeta `src` de tu espacio de trabajo, de modo que el proyecto se guarde en `src/`

`AceriaApp.Api`. Si ya trabajas con una solución, incorpora ambos proyectos a ella; los comandos de este manual se ejecutan desde la **raíz de la etapa**.

Progresión de archivos: en 2.2, 2.3 y 2.4 comienza con una copia del trabajo terminado del punto anterior, no con el proyecto de referencia que ya incorpora pasos futuros. Conserva un único `Program.cs` y una sola clase por nombre. Cuando se indique *añadir dentro*, no pegues otra clase completa: sitúa el nuevo método antes de su llave final. Si un paso propone analizar un bloque ya creado, **no lo dupliques**.

Resultado de aprendizaje: en cada paso realiza las cuatro acciones numeradas y comprueba el resultado antes de avanzar. Los pasos de diseño, inspección y depuración también exigen evidencia, aunque no modifiquen archivos. Los retos se realizan en copias aparte y no forman parte de las pruebas obligatorias de la etapa.

Punto 2.1 — Del diseño REST a los primeros endpoints GET

Proyecto comprobado: `PROYECTO/2.1-diseno-apis-rest`. El punto comienza estudiando el contrato de recursos y concluye con una ejecución verificable. Ninguna comprobación sustituye a un endpoint realmente implementado.

PASO 2.1.1 — Crear el proyecto ASP.NET Core Empty desde Visual Studio

Resultado de aprendizaje. Crear el proyecto ASP.NET Core Empty desde Visual Studio.

Procedimiento guiado.

1. En Visual Studio selecciona **Archivo > Nuevo > Proyecto**; busca «ASP.NET Core Empty» (C#), no la plantilla MVC ni Web API con controladores.
2. Ponle el nombre `AceriaApp.Api`, elige la carpeta de trabajo del punto 2.1 y selecciona **.NET 8.0**. Deja desactivada la autenticación; conserva HTTPS solo si vas a configurar el certificado local.
3. Pulsa **Crear**, abre `AceriaApp.Api.csproj` en el Explorador de soluciones y comprueba `<TargetFramework>net8.0</TargetFramework>`. Si no aparece .NET 8, instala ese SDK antes de seguir.
4. Ejecuta una compilación desde **Compilar > Compilar solución** o `dotnet build` en la carpeta del proyecto; identifica `Program.cs` como punto de entrada y guarda la solución de trabajo.

Por qué funciona así. El programa aún se está preparando: comprobar SDK, arranque y URI antes de introducir persistencia permite aislar errores de herramienta frente a errores de API.

Verificación concreta. Debes disponer de `src/AceriaApp.Api/AceriaApp.Api.csproj` con `net8.0`, un `Program.cs` generado por la plantilla y una compilación sin errores. No debe existir aún ningún endpoint de órdenes.

Si algo falla. Si falla el inicio, revisa que hayas seleccionado ASP.NET Core Empty y net8.0; no es necesario instalar controladores.

PASO 2.1.2 — Explorar Program.cs, Kestrel y el perfil de arranque

Resultado de aprendizaje. Explorar Program.cs, Kestrel y el perfil de arranque.

Procedimiento guiado.

1. Abre `Program.cs` y localiza `WebApplication.CreateBuilder(args)`, `builder.Build()` y `app.Run()`. Anota en qué orden se ejecutan.

2. En el selector de inicio, elige el perfil del proyecto `AceriaApp.Api` y pulsa **F5**; alternativamente usa `dotnet run --project src/AceriaApp.Api` desde la raíz de tu trabajo.
3. Observa en la ventana de salida la línea `Now listening on:` y copia la URL **real** (puerto incluido). Abre esa dirección en el navegador y distingue la ruta `/` de una ruta todavía inexistente.
4. Detén Kestrel con el botón **Detener** o `Ctrl+C`. Si la plantilla incluye un `MapGet("/")` de ejemplo, identifícalo; se retirará cuando sustituyas `Program.cs` por la API del taller.

Por qué funciona así. El programa aún se está preparando: comprobar SDK, arranque y URI antes de introducir persistencia permite aislar errores de herramienta frente a errores de API.

Verificación concreta. Registra la URL y puerto de escucha y demuestra que puedes iniciar y detener el servidor vacío. No interpretes un 404 en `/api/v1/ordenes` como error: ese endpoint todavía no existe.

Si algo falla. Si falla el inicio, revisa que hayas seleccionado ASP.NET Core Empty y net8.0; no es necesario instalar controladores.

PASO 2.1.3 — Dibujar los recursos y decidir las dos URIs GET

Resultado de aprendizaje. Dibujar los recursos y decidir las dos URIs GET.

Procedimiento guiado.

1. En una tabla de diseño escribe dos filas: `GET /api/v1/ordenes` (colección) y `GET /api/v1/ordenes/{id:guid}` (un miembro).
2. Para cada fila anota qué representa la URI, qué entrada necesita, qué JSON debe devolver y qué ocurriría si no hay registros o el GUID no existe.
3. Separa la identidad pública de una orden (`Guid Id`) de su representación (`OrdenDto`); razona por qué el cliente no debería ver obligatoriamente todos los campos de EF Core.
4. Antes de programar, comprueba que las dos URIs son sustantivos de recurso y que la plantilla `{id:guid}` exigirá un identificador con formato GUID.

Por qué funciona así. El programa aún se está preparando: comprobar SDK, arranque y URI antes de introducir persistencia permite aislar errores de herramienta frente a errores de API.

Verificación concreta. La tabla debe mostrar que una colección vacía se obtiene con 200 y `[]`; una orden inexistente con URI válida se responderá con 404. Aún no se realizan llamadas HTTP.

Si algo falla. Si falla el inicio, revisa que hayas seleccionado ASP.NET Core Empty y net8.0; no es necesario instalar controladores.

PASO 2.1.4 — Crear carpeta Modelo y entidad Orden

Resultado de aprendizaje. Crear carpeta Modelo y entidad Orden.

Procedimiento guiado.

1. En **Explorador de soluciones**, botón derecho sobre `AceriaApp.Api` > **Agregar > Nueva carpeta**; escribe `Modelo`.
2. Dentro de `Modelo`, usa **Agregar > Clase** y crea `Orden.cs`. Reemplaza el contenido generado por la clase completa del bloque C# de este paso, respetando `namespace AceriaApp.Api.Modelo`.
3. Comprueba uno por uno los seis datos: `Id`, `Cliente`, `Material`, `Cantidad`, `Estado` y `FechaCreacion`. Conserva la inicialización de Guid, estado y hora UTC.
4. Guarda con `Ctrl+S` y compila. Si el proyecto muestra un error de namespace, contrasta la ruta física `Modelo/Orden.cs` con la declaración `namespace`.

Archivo de trabajo: **`src/AceriaApp.Api/Modelo/Orden.cs`**.

```
namespace AceriaApp.Api.Modelo;
// INICIO PASO 2.1.4
public sealed class Orden
{
    public Guid Id { get; set; } = Guid.NewGuid();
    public string Cliente { get; set; } = string.Empty;
    public string Material { get; set; } = string.Empty;
    public int Cantidad { get; set; }
    public string Estado { get; set; } = "Pendiente";
    public DateTime FechaCreacion { get; set; } = DateTime.UtcNow;
}
// FIN PASO 2.1.4
```

Por qué funciona así. La entidad y el DTO cumplen funciones diferentes. La primera modela los datos persistidos y el segundo limita el contrato JSON que puede ver el consumidor.

Verificación concreta. `Orden.cs` debe compilar, contener las seis propiedades indicadas y dar `Estado == "Pendiente"` al crear `new Orden()`. Todavía no existe un controlador, repositorio ni DTO.

Si algo falla. Si no compilan Orden u OrdenDto, comprueba namespaces y tipos. No conviertas la entidad en el contrato JSON.

PASO 2.1.5 — Explicar ID Guid, Estado y FechaCreacion de la entidad

Resultado de aprendizaje. Explicar ID Guid, Estado y FechaCreacion de la entidad.

Procedimiento guiado.

1. Reabre `Modelo/Orden.cs`, identifica las expresiones `Guid.NewGuid()`, `"Pendiente"` y `DateTime.UtcNow` y explica qué valor crea cada una.
2. En el depurador, crea una instancia de `Orden` al llegar al paso donde se utiliza por primera vez (o examina el inicializador en el editor); comprueba qué campos quedan vacíos antes de recibir una petición.
3. Anota qué campos podrían establecerse desde `CrearOrdenRequest` más adelante y cuáles seguirán bajo control del servidor (`Id`, `Estado`, `FechaCreacion`).
4. No modifiques todavía la entidad ni dupliques su clase. Completa una pequeña tabla «propiedad → valor inicial → responsable» como evidencia conceptual.

Por qué funciona así. La entidad y el DTO cumplen funciones diferentes. La primera modela los datos persistidos y el segundo limita el contrato JSON que puede ver el consumidor.

Verificación concreta. La tabla diferencia los tres valores inicializados por servidor y los datos de negocio que se recibirán del cliente. El código de `Orden.cs` debe permanecer exactamente igual que al finalizar 2.1.4.

Si algo falla. Si no compilan Orden u OrdenDto, comprueba namespaces y tipos. No conviertas la entidad en el contrato JSON.

PASO 2.1.6 — Crear OrdenDto como contrato de salida

Resultado de aprendizaje. Crear OrdenDto como contrato de salida.

Procedimiento guiado.

1. En la carpeta `Modelo`, crea otro archivo **Clase** llamado `OrdenDto.cs`. Elimina la clase vacía sugerida por Visual Studio.
2. Escribe el `public sealed record OrdenDto(...)` del ejemplo del paso 2.1.6, con exactamente los seis parámetros públicos que se serializarán en JSON.
3. Comprueba el namespace `AceriaApp.Api.Modelo` y que no has añadido `DbSet<OrdenDto>` ni atributos de tabla: este tipo es un contrato de salida, no una entidad persistente.

4. Guarda y compila; no copies todavía el método `Desde`, porque corresponde a la ampliación del paso siguiente.

Archivo de trabajo: **`src/AceriaApp.Api/Modelo/OrdenDto.cs`**.

```
namespace AceriaApp.Api.Modelo;

public sealed record OrdenDto(
    Guid Id, string Cliente, string Material,
    int Cantidad, string Estado, DateTime FechaCreacion);
```

Por qué funciona así. La entidad y el DTO cumplen funciones diferentes. La primera modela los datos persistidos y el segundo limita el contrato JSON que puede ver el consumidor.

Verificación concreta. Existe un único archivo `OrdenDto.cs` con un record público de seis componentes. Compila y no contiene todavía ninguna segunda declaración de `OrdenDto`.

Si algo falla. Si no compilan Orden u OrdenDto, comprueba namespaces y tipos. No conviertas la entidad en el contrato JSON.

PASO 2.1.7 — Programar la conversión OrdenDto.Desde

Resultado de aprendizaje. Programar la conversión OrdenDto.Desde.

Procedimiento guiado.

1. Abre el `OrdenDto.cs` del paso anterior. Sustituye su `;` final por un cuerpo entre llaves `{ ... }`, sin conservar dos records con el mismo nombre.
2. Dentro del record añade exactamente el método `public static OrdenDto Desde(Orden x)` mostrado en el fragmento completo de este paso.
3. Relaciona en orden `x.Id`, `x.Cliente`, `x.Material`, `x.Cantidad`, `x.Estado` y `x.FechaCreacion` con los parámetros del record. No alteres la entidad original.
4. Guarda y compila. Revisa el mensaje del compilador si una propiedad se llama de modo distinto o si quedó un punto y coma antes del cuerpo.

Archivo de trabajo: **`src/AceriaApp.Api/Modelo/OrdenDto.cs`**.

```
namespace AceriaApp.Api.Modelo;

public sealed record OrdenDto(
    Guid Id, string Cliente, string Material,
    int Cantidad, string Estado, DateTime FechaCreacion)
{
    // PASO 2.1.7: transformar Orden en OrdenDto.
    public static OrdenDto Desde(Orden x) =>
        new(x.Id, x.Cliente, x.Material, x.Cantidad, x.Estado, x.FechaCreacion);
}
```

Por qué funciona así. La entidad y el DTO cumplen funciones diferentes. La primera modela los datos persistidos y el segundo limita el contrato JSON que puede ver el consumidor.

Verificación concreta. `OrdenDto.Desde(new Orden())` debe poder producir un `OrdenDto` sin conversión explícita de tipos ni duplicación del record.

Si algo falla. Si no compilan Orden u OrdenDto, comprueba namespaces y tipos. No conviertas la entidad en el contrato JSON.

PASO 2.1.8 — Instalar EF Core 8 para SQL Server

Resultado de aprendizaje. Instalar EF Core 8 para SQL Server.

Procedimiento guiado.

1. Sobre el proyecto `AceriaApp.Api` abre **Administrar paquetes NuGet** y busca `Microsoft.EntityFrameworkCore.SqlServer`.
2. Selecciona una versión de la rama **8.0.x** compatible con el proyecto (la referencia utiliza `8.0.21`), acepta los cambios y deja que NuGet restaure dependencias.
3. Comprueba en `AceriaApp.Api.csproj` que aparece el `PackageReference` a SQL Server. Como alternativa reproducible, desde la carpeta raíz de tu solución ejecuta `dotnet add src/AceriaApp.Api/AceriaApp.Api.csproj package Microsoft.EntityFrameworkCore.SqlServer --version 8.0.21`.
4. Ejecuta `dotnet restore src/AceriaApp.Api/AceriaApp.Api.csproj`. No añadas SQLite ni InMemory a la aplicación principal; InMemory se reservará al proyecto de tests.

Por qué funciona así. El acceso a datos se configura en infraestructura mediante EF Core y DI. LocalDB es la base efectiva de la API; EnsureCreated se usa solo durante el aprendizaje.

Verificación concreta. El `.csproj` contiene el proveedor `Microsoft.EntityFrameworkCore.SqlServer` versión 8.x y `dotnet restore` termina correctamente. Esto aún no prueba que LocalDB esté disponible.

Si algo falla. Si falla UseSqlServer, revisa la dependencia NuGet. Si LocalDB no responde, verifica la instancia MSSQLLocalDB.

PASO 2.1.9 — Crear AceriaDbContext y DbSet

Resultado de aprendizaje. Crear AceriaDbContext y DbSet.

Procedimiento guiado.

1. Crea la carpeta `Datos` dentro de `src/AceriaApp.Api` y añade el archivo `AceriaDbContext.cs`.
2. Escribe el código del paso 2.1.9: importa `AceriaApp.Api.Modelo`, importa `Microsoft.EntityFrameworkCore` y define el namespace `AceriaApp.Api.Datos`.
3. Haz que `AceriaDbContext` herede de `DbContext`, reciba `DbContextOptions<AceriaDbContext>` y exponga `DbSet<Orden> Ordenes => Set<Orden>()`.
4. Guarda y compila para confirmar que Visual Studio resuelve `DbContext`, `DbSet` y `Orden`. Deja `OnModelCreating` para el paso 2.1.10.

Archivo de trabajo: **`src/AceriaApp.Api/Datos/AceriaDbContext.cs`**.

```
using AceriaApp.Api.Modelo;
using Microsoft.EntityFrameworkCore;

namespace AceriaApp.Api.Datos;

public sealed class AceriaDbContext(
    DbContextOptions<AceriaDbContext> options) : DbContext(options)
{
    public DbSet<Orden> Ordenes => Set<Orden>();
}
```

Por qué funciona así. El acceso a datos se configura en infraestructura mediante EF Core y DI. LocalDB es la base efectiva de la API; EnsureCreated se usa solo durante el aprendizaje.

Verificación concreta. `AceriaDbContext.cs` compila y expone `Ordenes` como conjunto EF Core. Todavía no tiene reglas de longitud ni clave configuradas.

Si algo falla. Si falla UseSqlServer, revisa la dependencia NuGet. Si LocalDB no responde, verifica la instancia MSSQLLocalDB.

PASO 2.1.10 — Configurar OnModelCreating, clave y longitudes

Resultado de aprendizaje. Configurar OnModelCreating, clave y longitudes.

Procedimiento guiado.

1. Abre el `AceriaDbContext.cs` que creaste y sitúa el cursor **dentro de la clase**, justo antes de su llave final.
2. Inserta el método `protected override void OnModelCreating(ModelBuilder b)` que aparece en el fragmento. No crees otro contexto.
3. Comprueba `HasKey(x => x.Id)` y las reglas de `Cliente`, `Material` y `Estado`: longitudes 100, 100 y 24 respectivamente, todas obligatorias.
4. Guarda, ejecuta `dotnet build` y diferencia estas restricciones de modelo de las futuras reglas FluentValidation: aquí estás describiendo el almacenamiento.

Código de este paso. Añade el método al cuerpo del `AceriaDbContext` creado en 2.1.9; no borres el constructor ni `DbSet`. Comprueba que la clase final coincide con la referencia del punto 2.1.

```
// Dentro de AceriaDbContext, antes de su última llave:
protected override void OnModelCreating(ModelBuilder b)
{
    var e = b.Entity<Orden>();
    e.HasKey(x => x.Id);
    e.Property(x => x.Cliente).HasMaxLength(100).IsRequired();
    e.Property(x => x.Material).HasMaxLength(100).IsRequired();
    e.Property(x => x.Estado).HasMaxLength(24).IsRequired();
}
```

Por qué funciona así. El acceso a datos se configura en infraestructura mediante EF Core y DI. LocalDB es la base efectiva de la API; EnsureCreated se usa solo durante el aprendizaje.

Verificación concreta. El archivo final del contexto coincide en propiedades y reglas con `PROYECTO/2.1.../Datos/AceriaDbContext.cs`. Debe compilar, sin clases ni métodos duplicados.

Si algo falla. Si falla UseSqlServer, revisa la dependencia NuGet. Si LocalDB no responde, verifica la instancia MSSQLLocalDB.

PASO 2.1.11 — Configurar la cadena de conexión a LocalDB

Resultado de aprendizaje. Configurar la cadena de conexión a LocalDB.

Procedimiento guiado.

1. En `src/AceriaApp.Api/appsettings.json`, conserva la estructura de objeto JSON y crea la sección `ConnectionStrings` si no existe.
2. Añade la clave `AceriaApp` con el servidor `(localdb)\MSSQLLocalDB`, la base `AceriaAppM02`, `Trusted_Connection=True` y `TrustServerCertificate=True`. En JSON la barra invertida de LocalDB debe aparecer escapada como `\\`.
3. Comprueba que SQL Server Express LocalDB está instalado. Desde una consola de Windows ejecuta `sqllocaldb info MSSQLLocalDB`; si no está iniciada, usa `sqllocaldb start MSSQLLocalDB`.
4. Guarda el JSON, comprueba las comas y las comillas y no añadas usuarios ni contraseñas reales al repositorio.

Por qué funciona así. El acceso a datos se configura en infraestructura mediante EF Core y DI. LocalDB es la base efectiva de la API; EnsureCreated se usa solo durante el aprendizaje.

Verificación concreta. `builder.Configuration.GetConnectionString("AceriaApp")` deberá resolver la cadena correcta. El nombre `AceriaApp` es el de la clave de configuración; `AceriaAppM02` es el nombre de la base.

Si algo falla. Si falla UseSqlServer, revisa la dependencia NuGet. Si LocalDB no responde, verifica la instancia MSSQLLocalDB.

PASO 2.1.12 — Registrar AceriaDbContext mediante AddDbContext

Resultado de aprendizaje. Registrar AceriaDbContext mediante AddDbContext.

Procedimiento guiado.

1. Abre `Program.cs` y elimina el endpoint demostrativo `MapGet("/")` de la plantilla vacía, si existe. Conserva la estructura `CreateBuilder` → `Build` → `Run`.
2. Al comienzo importa `AceriaApp.Api.Datos`, `AceriaApp.Api.Modelo` y `Microsoft.EntityFrameworkCore`. Sitúa el cursor **después de `CreateBuilder(args)` y antes de `builder.Build()`**.
3. Añade únicamente `builder.Services.AddDbContext<AceriaDbContext>(...)` del bloque de este paso; `UseSqlServer` debe leer la cadena `AceriaApp` configurada en 2.1.11.
4. Guarda y compila. No pegues aquí `EnsureCreatedAsync`, porque esa inicialización se incorpora en el paso 2.1.13. No crees un segundo `builder` ni una segunda aplicación.

Archivo de trabajo: **`src/AceriaApp.Api/Program.cs`**.

```
// En Program.cs, entre var builder = WebApplication.CreateBuilder(args)
// y var app = builder.Build():
builder.Services.AddDbContext<AceriaDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("AceriaApp")));
```

Por qué funciona así. El acceso a datos se configura en infraestructura mediante EF Core y DI. LocalDB es la base efectiva de la API; EnsureCreated se usa solo durante el aprendizaje.

Verificación concreta. `Program.cs` compila con una sola llamada `AddDbContext<AceriaDbContext>` anterior a `Build`, y la aplicación sigue arrancando aunque aún no tenga endpoints REST.

Si algo falla. Si falla UseSqlServer, revisa la dependencia NuGet. Si LocalDB no responde, verifica la instancia MSSQLLocalDB.

PASO 2.1.13 — Crear la base didáctica solo en Development

Resultado de aprendizaje. Crear la base didáctica solo en Development.

Procedimiento guiado.

1. En `Program.cs`, busca exactamente la línea `var app = builder.Build();` y coloca el cursor **después de ella** y antes de cualquier `app.MapGet`.
2. Inserta el bloque de `if (app.Environment.IsDevelopment() && !...)` con `CreateScope()`, `GetRequiredService<AceriaDbContext>()` y `EnsureCreatedAsync()` del ejemplo.
3. En Visual Studio selecciona el perfil Development y comprueba que LocalDB esté iniciada. Arranca la API y revisa en el Explorador de objetos SQL Server si se creó `AceriaAppM02`.
4. Detén el servidor. Explica por qué los tests establecerán `SkipDatabaseInitialization=true` y por qué `EnsureCreated` no sustituye a las migraciones de producción.

Código de este paso. Este bloque va después de `builder.Build()` y antes de mapear los endpoints. Crea la base de aprendizaje solo en Development; la opción `SkipDatabaseInitialization` se utiliza en los tests de integración.

```
// En Program.cs, justo después de var app = builder.Build():
if (app.Environment.IsDevelopment()
    && !app.Configuration.GetValue<bool>("SkipDatabaseInitialization"))
{
    using var scope = app.Services.CreateScope();
```

```
    await scope.ServiceProvider.GetRequiredService<AceriaDbContext>()
        .Database.EnsureCreatedAsync();
}
```

Por qué funciona así. El acceso a datos se configura en infraestructura mediante EF Core y DI. LocalDB es la base efectiva de la API; EnsureCreated se usa solo durante el aprendizaje.

Verificación concreta. La aplicación Development arranca sin error de conexión y la base `AceriaAppM02` existe en LocalDB. El bloque aparece una sola vez y no se ejecuta en Production.

Si algo falla. Si falla UseSqlServer, revisa la dependencia NuGet. Si LocalDB no responde, verifica la instancia MSSQLLocalDB.

PASO 2.1.14 — Escribir el endpoint GET de colección

Resultado de aprendizaje. Escribir el endpoint GET de colección.

Procedimiento guiado.

1. En el mismo `Program.cs`, antes de `app.Run()`, pega el único `app.MapGet("/api/v1/ordenes", ...)` del bloque 2.1.14.
2. Asegúrate de que la lambda recibe `AceriaDbContext db` y `CancellationToken ct`, consulta `db.Ordenes`, ordena por `FechaCreacion` y transforma cada entidad mediante `OrdenDto.Desde`.
3. Añade al final de la operación `WithName("ListarOrdenes")` y `WithTags("Ordenes")`. No escribas un segundo endpoint para la misma ruta.
4. Compila y ejecuta. En esta primera versión todavía no se utiliza `AsNoTracking`; lo añadirás al endpoint **existente** en el próximo paso.

Archivo de trabajo: **`src/AceriaApp.Api/Program.cs`**.

```
app.MapGet("/api/v1/ordenes",
    async (AceriaDbContext db, CancellationToken ct) =>
{
    var ordenes = await db.Ordenes
        .OrderBy(x => x.FechaCreacion).ToListAsync(ct);
    return Results.Ok(ordenes.Select(OrdenDto.Desde));
})
.WithName("ListarOrdenes").WithTags("Ordenes");
```

Por qué funciona así. El verbo GET no modifica recursos. MapGet recibe dependencias y parámetros, consulta la orden y traduce el resultado a 200 o 404 con DTOs.

Verificación concreta. Una petición GET a `/api/v1/ordenes` debe devolver 200 y JSON de colección. No debe existir otra ruta GET de colección duplicada en `Program.cs`.

Si algo falla. Si aparece 404, revisa Guid, la plantilla {id:guid} y la existencia de la orden; en colección vacía corresponde 200.

PASO 2.1.15 — Utilizar AsNoTracking y proyectar a DTO

Resultado de aprendizaje. Utilizar AsNoTracking y proyectar a DTO.

Procedimiento guiado.

1. Abre el `MapGet` de colección del paso 2.1.14 y localiza la consulta que comienza con `var ordenes = await db.Ordenes`.
2. Sustituye **solo** esa instrucción por la consulta mostrada en 2.1.15, incorporando `.AsNoTracking()` antes de `OrderBy`.
3. No cambies el nombre de ruta, el delegado, `Results.Ok`, ni la transformación a `OrdenDto`; conserva el resto del manejador sin duplicarlo.

4. Guarda, compila y ejecuta el GET. Comprueba que devuelve el mismo JSON, porque `AsNoTracking` cambia el seguimiento de entidades, no el contrato HTTP.

Código de este paso. Modifica la consulta existente y verifica que la ruta, el tipo de respuesta y la ordenación siguen siendo los mismos. Esta consulta es la versión final del endpoint de colección.

```
// En el MapGet de 2.1.14, sustituye solo la consulta:
var ordenes = await db.Ordenes.AsNoTracking()
    .OrderBy(x => x.FechaCreacion).ToListAsync(ct);
// Mantén la llamada: return Results.Ok(ordenes.Select(OrdenDto.Desde));
```

Por qué funciona así. El verbo GET no modifica recursos. MapGet recibe dependencias y parámetros, consulta la orden y traduce el resultado a 200 o 404 con DTOs.

Verificación concreta. El proyecto compila y el GET conserva 200 y los mismos campos DTO. En el código hay un solo `MapGet` de colección con `AsNoTracking()`.

Si algo falla. Si aparece 404, revisa Guid, la plantilla {id:guid} y la existencia de la orden; en colección vacía corresponde 200.

PASO 2.1.16 — Ejecutar GET y verificar 200 con lista vacía

Resultado de aprendizaje. Ejecutar GET y verificar 200 con lista vacía.

Procedimiento guiado.

1. Arranca `AceriaApp.Api` en Development y toma la URL publicada en `Now listening on:`. No presupongas que siempre es el puerto 5082.
2. Abre o crea `AceriaApp.http` en la raíz de tu carpeta de trabajo. Ajusta `@Host` a esa URL y escribe `GET {{Host}}/api/v1/ordenes`.
3. Pulsa **Send Request** sobre esa línea (cliente HTTP de Visual Studio) o usa `curl` contra el mismo host. Observa la línea de estado, la cabecera `Content-Type` y el cuerpo.
4. Si la tabla aún está vacía, verifica que el JSON sea `[]`. Si ya hay datos de otro intento, usa una base de pruebas limpia o explica por qué aparecen elementos.

Por qué funciona así. El verbo GET no modifica recursos. MapGet recibe dependencias y parámetros, consulta la orden y traduce el resultado a 200 o 404 con DTOs.

Verificación concreta. Evidencia: captura o copia de la respuesta con `HTTP/1.1 200` y un array JSON (`[]` en base vacía). Un conjunto vacío no produce 404.

Si algo falla. Si aparece 404, revisa Guid, la plantilla {id:guid} y la existencia de la orden; en colección vacía corresponde 200.

PASO 2.1.17 — Escribir el GET individual con

Resultado de aprendizaje. Escribir el GET individual con {id:guid}.

Procedimiento guiado.

1. En `Program.cs`, debajo del GET de colección y antes de `app.Run()`, añade una segunda **ruta distinta**: `/api/v1/ordenes/{id:guid}`.
2. Copia la lambda completa del bloque: recibe `Guid id`, obtiene el contexto por DI, ejecuta `FirstOrDefaultAsync` y convierte la entidad a `OrdenDto` si existe.
3. Define `IResult response` y conserva ambas ramas, `Results.Ok(...)` y `Results.NotFound()`. Termina con `WithName("ObtenerOrdenPorId")` y `WithTags("Ordenes")`.
4. Guarda, compila y prueba el GET con `00000000-0000-0000-0000-000000000001` si no existe esa orden.

Archivo de trabajo: **`src/AceriaApp.Api/Program.cs`**.

```
app.MapGet("/api/v1/ordenes/{id:guid}",
    async (Guid id, AceriaDbContext db) =>
{
    var orden = await db.Ordenes.AsNoTracking()
        .FirstOrDefaultAsync(x => x.Id == id);
    IResult response = orden is null
        ? Results.NotFound()
        : Results.Ok(OrdenDto.Desde(orden));
    return response;
})
.WithName("ObtenerOrdenPorId").WithTags("Ordenes");
```

Por qué funciona así. El verbo GET no modifica recursos. MapGet recibe dependencias y parámetros, consulta la orden y traduce el resultado a 200 o 404 con DTOs.

Verificación concreta. La segunda ruta responde 404 para un GUID bien formado ausente. Debe existir **una sola** declaración de cada GET en `Program.cs`.

Si algo falla. Si aparece 404, revisa Guid, la plantilla {id:guid} y la existencia de la orden; en colección vacía corresponde 200.

PASO 2.1.18 — Devolver NotFound cuando no existe la orden

Resultado de aprendizaje. Devolver NotFound cuando no existe la orden.

Procedimiento guiado.

1. No añadas código nuevo: abre el GET individual del paso 2.1.17 y marca las líneas `FirstOrDefaultAsync` y el condicional `orden is null`.
2. Establece un punto de interrupción en la instrucción que construye `IResult response` y ejecuta con F5.
3. Envía desde el archivo HTTP una petición con un GUID válido que no exista. Inspecciona en el depurador que `orden == null`.
4. Continúa la ejecución y comprueba que la respuesta es 404 sin un DTO inventado. Si el identificador sí existe, la otra rama devolverá 200 con la representación.

Por qué funciona así. El verbo GET no modifica recursos. MapGet recibe dependencias y parámetros, consulta la orden y traduce el resultado a 200 o 404 con DTOs.

Verificación concreta. Demuestra que el manejador, no la base de datos ni Swagger, decide devolver 404 cuando la consulta no encuentra una orden.

Si algo falla. Si aparece 404, revisa Guid, la plantilla {id:guid} y la existencia de la orden; en colección vacía corresponde 200.

PASO 2.1.19 — Comprobar diferencias entre ruta incorrecta y recurso ausente

Resultado de aprendizaje. Comprobar diferencias entre ruta incorrecta y recurso ausente.

Procedimiento guiado.

1. Prepara dos URL diferentes en `AceriaApp.http`: una terminada en `/no-es-guid` y otra terminada en `/00000000-0000-0000-0000-000000000001`.
2. Pon un punto de interrupción al entrar en el delegado del GET por id y ejecuta primero la URL con texto no GUID.
3. Observa que la restricción de ruta `{id:guid}` impide que entre en el delegado. Ejecuta después la URL con GUID bien formado, pero inexistente.
4. Comprueba que ahora sí se entra en el delegado, se consulta EF Core y se obtiene `NotFound`. Documenta las dos causas aunque el código HTTP externo pueda ser 404.

Por qué funciona así. El verbo GET no modifica recursos. MapGet recibe dependencias y parámetros, consulta la orden y traduce el resultado a 200 o 404 con DTOs.

Verificación concreta. Tabla con «URL», «¿entra en el endpoint?», «origen del 404». Debe mostrar NO para formato inválido y SÍ para GUID válido ausente.

Si algo falla. Si aparece 404, revisa Guid, la plantilla {id:guid} y la existencia de la orden; en colección vacía corresponde 200.

PASO 2.1.20 — Crear AceriaApp.http y enviar peticiones desde Visual Studio

Resultado de aprendizaje. Crear AceriaApp.http y enviar peticiones desde Visual Studio.

Procedimiento guiado.

1. En la raíz de la carpeta del punto 2.1, crea `AceriaApp.http` con **Agregar > Nuevo elemento > archivo HTTP**, o un archivo de texto con extensión `.http`.
2. Escribe `@Host = http://localhost:PUERTO_REAL` usando el puerto visto al arrancar. Añade una petición GET de colección, una línea `###` y una petición GET de detalle con un GUID de ejemplo.
3. Arranca la API y ejecuta las dos peticiones con **Send Request**. Guarda junto a cada una el resultado esperado: 200 con array para colección y 404 para recurso ausente.
4. Abre `Program.cs` en el depurador y comprueba qué dependencia se inyecta; realiza el caso práctico completo de lectura EF Core situado después de esta actividad.

Por qué funciona así. El verbo GET no modifica recursos. MapGet recibe dependencias y parámetros, consulta la orden y traduce el resultado a 200 o 404 con DTOs.

Verificación concreta. El archivo `.http` queda guardado y reproduce dos respuestas observables de la API real, sin inventar un puerto ni pedir Swagger en una etapa que aún no lo incluye.

Si algo falla. Si aparece 404, revisa Guid, la plantilla {id:guid} y la existencia de la orden; en colección vacía corresponde 200.

Caso práctico completo — Del navegador hasta una consulta EF Core

Este caso demuestra el trayecto completo de una petición GET antes de añadir pruebas. Abre `src/AceriaApp.Api/Program.cs` y coloca un punto de interrupción justo dentro de `app.MapGet("/api/v1/ordenes", ...)`. Ejecuta la API con F5 usando el perfil Development y abre `AceriaApp.http`. Envía `GET /api/v1/ordenes`. Cuando Visual Studio se detenga, inspecciona el argumento `AceriaDbContext db`. No lo ha creado el cliente HTTP: lo resolvió el contenedor de dependencias a partir de `AddDbContext`, configurado antes de construir la aplicación.

Continúa paso por paso. `db.Ordenes` representa la tabla de órdenes; `AsNoTracking()` indica que EF Core no va a mantener seguimiento de cambios en las entidades recuperadas. Esta opción resulta apropiada cuando no se pretende modificar esas mismas entidades. `OrderBy(x => x.FechaCreacion)` establece un orden reproducible de presentación, y `ToListAsync` ejecuta la consulta. Si no hay filas, el resultado será una lista vacía, que debe traducirse en un estado 200 con `[]` en el cuerpo JSON.

La llamada `OrdenDto.Desde` realiza el límite entre persistencia y representación. El cliente no recibe una instancia del contexto ni detalles sobre una tabla; recibe un objeto JSON con los campos públicos de la orden. La ruta de detalle añade una comprobación: si el Guid es válido pero la consulta no encuentra ninguna fila, el servidor responde 404. Comprueba esta situación con el identificador inexistente del archivo HTTP. Si recibes una excepción de conexión en lugar de 200 o 404, revisa que LocalDB está instalado y que la cadena de conexión apunta a `(localdb)\MSSQLLocalDB`.

Demostración de aprendizaje: explica por qué `Results.Ok` y `Results.NotFound` pertenecen al contrato HTTP y no al modelo de dominio. Cambia temporalmente una ruta en Program.cs y observa cómo deja de coincidir la petición anterior; restaura el código y vuelve a comprobarlo.

PASO 2.1.21 — Crear el proyecto xUnit de pruebas HTTP

Resultado de aprendizaje. Crear el proyecto xUnit de pruebas HTTP.

Procedimiento guiado.

1. En el Explorador de soluciones, crea un proyecto **xUnit Test Project** llamado `AceriaApp.Api.Tests` bajo `tests`, con destino .NET 8, o ejecuta los comandos PowerShell del bloque.
2. Agrega una referencia **de proyecto** desde los tests a `src/AceriaApp.Api/AceriaApp.Api.csproj`, no una referencia a una DLL copiada.
3. Instala en el proyecto de tests `Microsoft.AspNetCore.Mvc.Testing` y `Microsoft.EntityFrameworkCore.InMemory`, ambos 8.x. Comprueba que no se han añadido a la API de producción.
4. Elimina el archivo `UnitTest1.cs` generado por defecto en la plantilla xUnit si existe, porque el laboratorio entregará después tres pruebas específicas. Añade el proyecto de tests a la solución y ejecuta `dotnet restore`.

Archivo de trabajo: **`tests/AceriaApp.Api.Tests/ApiFactory.cs`**.

```
# Desde la carpeta raíz del punto 2.1:
dotnet new xunit -n AceriaApp.Api.Tests -o tests/AceriaApp.Api.Tests --framework net8.0
dotnet add tests/AceriaApp.Api.Tests/AceriaApp.Api.Tests.csproj reference src/AceriaApp.Api/
AceriaApp.Api.csproj
dotnet add tests/AceriaApp.Api.Tests/AceriaApp.Api.Tests.csproj package
Microsoft.AspNetCore.Mvc.Testing --version 8.0.21
dotnet add tests/AceriaApp.Api.Tests/AceriaApp.Api.Tests.csproj package
Microsoft.EntityFrameworkCore.InMemory --version 8.0.21
```

Por qué funciona así. WebApplicationFactory prueba la aplicación como servidor HTTP: permite distinguir compilación C# de comportamiento observable del enrutamiento y la serialización.

Verificación concreta. Existe `tests/AceriaApp.Api.Tests/AceriaApp.Api.Tests.csproj` con referencia a la API y los paquetes de integración. No debe quedar un test de plantilla que altere el recuento de 3 pruebas.

Si algo falla. Si falla ApiFactory, comprueba Program partial y el registro de DbContext con InMemory solo para tests.

PASO 2.1.22 — Configurar WebApplicationFactory y base aislada para tests

Resultado de aprendizaje. Configurar WebApplicationFactory y base aislada para tests.

Procedimiento guiado.

1. En la carpeta del proyecto de tests, crea `ApiFactory.cs` y pega íntegro el código C# de este paso. Verifica que la clase hereda de `WebApplicationFactory<Program>`.
2. Abre el `Program.cs` de la API y, **después de** **`app.Run();`**, añade `public partial class Program {}` para que el host de pruebas pueda localizar el punto de entrada.
3. En `ConfigureTestServices`, comprueba que se elimina `DbContextOptions<AceriaDbContext>` y se registra una base **InMemory** exclusiva de los tests, con un nombre aleatorio por factoría.
4. Comprueba la opción `SkipDatabaseInitialization=true`: esta configuración debe evitar que los tests necesiten LocalDB. Guarda y compila el proyecto de tests.

Archivo de trabajo: **`tests/AceriaApp.Api.Tests/ApiFactory.cs`**.

```
using AceriaApp.Api.Datos;
using Microsoft.AspNetCore.Hosting;
using Microsoft.AspNetCore.Mvc.Testing;
using Microsoft.AspNetCore.TestHost;
```

```
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.DependencyInjection.Extensions;
namespace AceriaApp.Api.Tests;
// INICIO PASO 2.1.22: HTTP real; base en memoria SOLO en tests automatizados.
public sealed class ApiFactory:WebApplicationFactory<Program>
{
    private readonly string _databaseName="Tests-"+Guid.NewGuid();
    protected override void ConfigureWebHost(IWebHostBuilder builder)
    {
        builder.UseEnvironment("Development");
        builder.UseSetting("SkipDatabaseInitialization","true");
        builder.ConfigureTestServices(services=>
        {
            services.RemoveAll<DbContextOptions<AceriaDbContext>>();
            services.AddDbContext<AceriaDbContext>(opt=>
                opt.UseInMemoryDatabase(_databaseName));
        });
    }
}
```

Por qué funciona así. WebApplicationFactory prueba la aplicación como servidor HTTP: permite distinguir compilación C# de comportamiento observable del enrutamiento y la serialización.

Verificación concreta. `ApiFactory` compila y arranca el host de pruebas sin instalar SQL Server. La aplicación habitual sigue utilizando LocalDB; solo el host de tests sustituye el proveedor.

Si algo falla. Si falla ApiFactory, comprueba Program partial y el registro de DbContext con InMemory solo para tests.

PASO 2.1.23 — Probar 200, 404 y DTO mediante GetTests

Resultado de aprendizaje. Probar 200, 404 y DTO mediante GetTests.

Procedimiento guiado.

1. Dentro del proyecto de tests crea `GetTests.cs` con los imports y la clase completa que aparece en el bloque C#.
2. Comprueba los tres métodos: `ListaVacia_Es200`, `RecursoInexistente_Es404` y `RecursoExistente_ExponeDto`; cada uno debe instanciar su factoría aislada.
3. En el tercer test observa la inserción mediante el contexto de prueba, `SaveChangesAsync`, el GET y la deserialización posterior a `OrdenDto`.
4. Ejecuta `dotnet test tests/AceriaApp.Api.Tests/AceriaApp.Api.Tests.csproj` desde la raíz del punto; abre el Explorador de pruebas si lo prefieres y comprueba nombres y resultados.

Archivo de trabajo: `tests/AceriaApp.Api.Tests/GetTests.cs`.

```
using System.Net;
using System.Net.Http.Json;
using AceriaApp.Api.Datos;
using AceriaApp.Api.Modelo;
using Microsoft.Extensions.DependencyInjection;
using Xunit;
namespace AceriaApp.Api.Tests;
// INICIO PASO 2.1.23: dos peticiones HTTP, código y DTO.
public class GetTests
{
    [Fact] public async Task ListaVacia_Es200()
    {
        using var f=new ApiFactory();
        var r=await f.CreateClient().GetAsync("/api/v1/ordenes");
        Assert.Equal(HttpStatusCode.OK,r.StatusCode);
    }
    [Fact] public async Task RecursoInexistente_Es404()
    {
```

```
        using var f=new ApiFactory();
        var r=await f.CreateClient().GetAsync("/api/v1/ordenes/"+Guid.NewGuid());
        Assert.Equal(HttpStatusCode.NotFound,r.StatusCode);
    }
    [Fact] public async Task RecursoExistente_ExponeDto()
    {
        using var f=new ApiFactory();
        var id=Guid.NewGuid();
        using(var scope=f.Services.CreateScope())
        {
            var db=scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
            db.Ordenes.Add(new Orden{Id=id,Cliente="ACX-001",Material="CHAPA",Cantidad=10});
            await db.SaveChangesAsync();
        }
        var r=await f.CreateClient().GetAsync("/api/v1/ordenes/"+id);
        var dto=await r.Content.ReadFromJsonAsync<OrdenDto>();
        Assert.Equal(HttpStatusCode.OK,r.StatusCode);
        Assert.Equal(id,dto?.Id);
    }
}
```

Por qué funciona así. WebApplicationFactory prueba la aplicación como servidor HTTP: permite distinguir compilación C# de comportamiento observable del enrutamiento y la serialización.

Verificación concreta. Debe indicar **3 tests superados, 0 fallidos**. Un fallo de `WebApplicationFactory` obliga a revisar `Program partial`, referencia de proyecto y configuración InMemory.

Si algo falla. Si falla ApiFactory, comprueba Program partial y el registro de DbContext con InMemory solo para tests.

PASO 2.1.24 — Reto: incorporar consulta por cliente y explicar su URI

Resultado de aprendizaje. Reto: incorporar consulta por cliente y explicar su URI.

Procedimiento guiado.

1. Este es un **reto opcional**: realiza una copia separada de tu solución 2.1 para no alterar el estado de referencia que utilizará 2.2.
2. En el GET de colección acepta un parámetro opcional `string? cliente`, filtra mediante `Where` solo si se ha enviado y conserva `AsNoTracking` y la respuesta DTO.
3. Crea una petición HTTP con `?cliente=ACX-001` y otra con un cliente inexistente; la segunda debe devolver 200 con `[]`, no 404.
4. Añade una prueba xUnit específica del filtro, ejecútala y documenta qué diferencia hay entre filtrar colección y consultar un recurso por GUID.

Por qué funciona así. El filtrado de la colección conserva la idea de recurso y añade una consulta con parámetros, sin transformar el endpoint en una operación RPC.

Verificación concreta. La solución ampliada filtra sin cambiar el contrato de los GET básicos. Este reto no debe introducirse obligatoriamente en el proyecto base de la etapa 2.2.

Si algo falla. Si el filtro no cambia el resultado, revisa el nombre cliente en la query y la condición Where.

Caso práctico completo — Probar una API de lectura sin depender de una base instalada en CI

Abre el proyecto de pruebas y localiza `ApiFactory.cs`. Esta clase deriva de `WebApplicationFactory<Program>` y arranca una aplicación real de ASP.NET Core en memoria. La aplicación del alumno sigue conectándose a LocalDB cuando se ejecuta en Windows; exclusivamente dentro de la fábrica de tests se sustituye el proveedor por EF Core InMemory. Esa decisión mantiene

las pruebas reproducibles y permite que se ejecuten en un runner Linux, pero no autoriza a utilizar InMemory como sustituto de SQL Server para verificar todo su comportamiento.

El primer test consulta la colección vacía y espera 200; el segundo construye un Guid nuevo, lo incorpora a la ruta de detalle y espera 404. Para comprobar el caso positivo se introduce una entidad Orden en la base de la prueba, se confirma SaveChangesAsync y después se envía un GET al mismo identificador. La respuesta se deserializa a OrdenDto. El test debe contrastar al menos el identificador y uno de los campos, porque comprobar solamente un 200 no garantiza que haya aparecido la orden correcta.

Ejecuta desde la terminal integrada `dotnet test` indicando el proyecto `tests/AceriaApp.Api.Tests`. Si un test que insertó una entidad recibe 404, revisa que el mismo contexto de la fábrica comparta base y nombre durante la inserción y el GET. Si el código compila pero el servidor de pruebas no arranca, comprueba que el programa expone `public partial class Program` y que el proyecto de tests referencia la API.

Reto de cierre del punto: escribe una prueba para el filtro opcional por cliente de la colección. Inserta dos órdenes de clientes diferentes, haz la consulta con un parámetro `cliente` y compara los identificadores. La prueba debe rechazar una implementación que devuelva todas las filas.

Matriz de comprobación del punto

Comportamiento	Resultado esperado
`GET /api/v1/ordenes`	200 + JSON
`GET /api/v1/ordenes/{id}`	200 / 404

Reto integrador. Reconstruye sin consultar la solución el último endpoint o validación practicada y añade una prueba HTTP que demuestre su resultado. Describe el recurso, el verbo, el contrato y el código devuelto. Si no puedes reproducir una respuesta, vuelve al primer paso que introdujo el comportamiento.

Punto 2.2 — Recursos, verbos, persistencia y respuestas HTTP

Proyecto comprobado: `PROYECTO/2.2-recursos-verbos-codigos`. El punto comienza estudiando el contrato de recursos y concluye con una ejecución verificable. Ninguna comprobación sustituye a un endpoint realmente implementado.

PASO 2.2.1 — Recuperar el estado GET funcional sin modificarlo

Resultado de aprendizaje. Recuperar el estado GET funcional sin modificarlo.

Procedimiento guiado.

1. Cierra la API del punto 2.1 y copia tu **solución de trabajo terminada** a una carpeta nueva para 2.2, conservando `src`, `tests` y `AceriaApp.http`. No copies el proyecto de referencia 2.2, porque ya contiene lo que vas a aprender.
2. Abre la copia 2.2 en Visual Studio y comprueba que no se están ejecutando simultáneamente dos servidores en el mismo puerto.
3. Ejecuta el proyecto de tests heredado y verifica sus tres casos de GET. Después arranca la API y prueba de nuevo colección 200 y recurso inexistente 404.
4. Anota el estado inicial y conserva una copia de seguridad del punto 2.1; los nuevos endpoints se añadirán progresivamente en `Program.cs`.

Por qué funciona así. El diseño de las rutas y los contratos evita recibir propiedades de servidor desde JSON. El resultado público se devuelve mediante OrdenDto, no directamente desde la entidad.

Verificación concreta. La etapa 2.2 parte de la API de lectura de 2.1 sin regresiones: 3 tests y las dos peticiones GET siguen correctos.

Si algo falla. Si el JSON puede cambiar el ID o Estado, el contrato de entrada está exponiendo datos internos; utiliza CrearOrdenRequest.

PASO 2.2.2 — Diseñar la tabla verbo, URI, petición y respuesta

Resultado de aprendizaje. Diseñar la tabla verbo, URI, petición y respuesta.

Procedimiento guiado.

1. Crea en tus notas una tabla con columnas **verbo, URI, JSON de entrada, respuesta y efecto persistente**.
2. Escribe una fila por cada operación prevista: GET de colección, GET individual, POST colección, PUT individual, PATCH `/cantidad`, DELETE individual y POST `/finalizar`.
3. Anota los éxitos previstos: 200 para GET, PATCH y finalizar; 201 + `Location` para creación; 204 para PUT y DELETE.
4. Añade los casos alternativos 400 (entrada inválida), 404 (id inexistente), 409 (estado incompatible) y 422 (capacidad del taller). Marca cuáles pertenecen a pasos siguientes para no implementarlos todos todavía.

Por qué funciona así. El diseño de las rutas y los contratos evita recibir propiedades de servidor desde JSON. El resultado público se devuelve mediante OrdenDto, no directamente desde la entidad.

Verificación concreta. Tu tabla debe enumerar las siete operaciones reales del proyecto de referencia 2.2 y distinguir los resultados HTTP de los cambios almacenados en `Ordenes`.

Si algo falla. Si el JSON puede cambiar el ID o Estado, el contrato de entrada está exponiendo datos internos; utiliza CrearOrdenRequest.

PASO 2.2.3 — Separar contrato de entrada y DTO de salida

Resultado de aprendizaje. Separar contrato de entrada y DTO de salida.

Procedimiento guiado.

1. Abre la entidad `Modelo/Orden.cs` y el contrato de salida `Modelo/OrdenDto.cs` creados en 2.1.
2. Crea la carpeta `Contratos` e identifica **qué puede escribir el cliente**: `Cliente`, `Material` y `Cantidad` para crear o sustituir, y solo `Cantidad` para PATCH.
3. Dibuja dos contratos de entrada: `CrearOrdenRequest(string Cliente, string Material, int Cantidad)` y `ActualizarCantidadRequest(int Cantidad)`. Compáralos con `OrdenDto`, que además publica id, estado y fecha.
4. Justifica por qué el endpoint POST no debería aceptar libremente un `Orden` completo con un `Id` proporcionado por el cliente.

Por qué funciona así. El diseño de las rutas y los contratos evita recibir propiedades de servidor desde JSON. El resultado público se devuelve mediante OrdenDto, no directamente desde la entidad.

Verificación concreta. Debe quedar una separación explícita entre entidad persistente, contrato de entrada y DTO de respuesta; la clase `Orden` existente no se modifica.

Si algo falla. Si el JSON puede cambiar el ID o Estado, el contrato de entrada está exponiendo datos internos; utiliza CrearOrdenRequest.

PASO 2.2.4 — Crear CrearOrdenRequest y ActualizarCantidadRequest

Resultado de aprendizaje. Crear CrearOrdenRequest y ActualizarCantidadRequest.

Procedimiento guiado.

1. En `src/AceriaApp.Api/Contratos` crea `ContratosOrden.cs`; si Visual Studio propone una clase vacía, sustitúyela por el contenido del fragmento C#.
2. Declara el namespace `AceriaApp.Api.Contratos` y **dos records**: `CrearOrdenRequest` y `ActualizarCantidadRequest`, exactamente como en el ejemplo.
3. Guarda el archivo e importa este namespace en `Program.cs` antes de registrar los endpoints que recibirán esos tipos.
4. Compila el proyecto y comprueba que el compilador reconoce ambos records sin mezclar sus propiedades con `OrdenDto`.

Archivo de trabajo: **`src/AceriaApp.Api/Contratos/ContratosOrden.cs`**.

```
namespace AceriaApp.Api.Contratos;
// INICIO PASO 2.2.4
public sealed record CrearOrdenRequest(string Cliente,string Material,int Cantidad);
public sealed record ActualizarCantidadRequest(int Cantidad);
// FIN PASO 2.2.4
```

Por qué funciona así. El diseño de las rutas y los contratos evita recibir propiedades de servidor desde JSON. El resultado público se devuelve mediante OrdenDto, no directamente desde la entidad.

Verificación concreta. El archivo contiene dos tipos públicos y ningún DTO de EF Core. Compilación correcta, sin duplicar namespace ni modificar los GET existentes.

Si algo falla. Si el JSON puede cambiar el ID o Estado, el contrato de entrada está exponiendo datos internos; utiliza CrearOrdenRequest.

PASO 2.2.5 — Programar MapPost y recibir JSON de una orden

Resultado de aprendizaje. Programar MapPost y recibir JSON de una orden.

Procedimiento guiado.

1. Abre `Program.cs`, añade `using AceriaApp.Api.Contratos;` al comienzo y sitúate **después de los GET y antes de `app.Run()`**.
2. Inserta **una sola vez** el `MapPost("/api/v1/ordenes", ...)` mostrado en el bloque C#. Conserva completos los parámetros `CrearOrdenRequest request` y `AceriaDbContext db`.
3. Revisa las tres ramas del handler: rechazo provisional de campos vacíos, límite de cantidad superior a 500 y camino válido de creación.
4. Guarda y compila. Los pasos 2.2.6–2.2.10 analizarán, ejecutarán y contrastarán esas ramas; no debes crear más versiones del POST.

Archivo de trabajo: **`src/AceriaApp.Api/Program.cs`**.

```
// INICIO PASO 2.2.5: crear una orden, persistirla, devolver 201 + Location.
app.MapPost("/api/v1/ordenes",async (CrearOrdenRequest request,AceriaDbContext db)=>
{
    if(string.IsNullOrWhiteSpace(request.Cliente)||string.IsNullOrWhiteSpace(request.Material)||
request.Cantidad<=0)
        return Results.BadRequest(new {error="Cliente, material y cantidad son obligatorios."});
    if(request.Cantidad>500)
        return Results.Problem(title:"El taller solo admite lotes de hasta 500.",
            statusCode:StatusCodes.Status422UnprocessableEntity);
    var orden=new Orden{Cliente=request.Cliente,Material=request.Material,Cantidad=request.Cantidad};
    db.Ordenes.Add(orden);
    await db.SaveChangesAsync();
```

```
    return Results.CreatedAtRoute("ObtenerOrdenPorId",new{id=orden.Id},OrdenDto.Desde(orden));
})
.WithName("CrearOrden").WithTags("Ordenes")


;
```

Por qué funciona así. El POST crea un recurso y debe confirmar la inserción mediante SaveChangesAsync. CreatedAtRoute devuelve 201 y un enlace Location utilizable mediante GET.

Verificación concreta. Existe exactamente un POST de colección, compila y acepta JSON con `cliente`, `material` y `cantidad`; un request de negocio correcto alcanza el camino de creación.

Si algo falla. Si el POST no persiste, verifica SaveChangesAsync; si Location falta, revisa CreatedAtRoute y el nombre de GET.

PASO 2.2.6 — Persistir la orden con Add y SaveChangesAsync

Resultado de aprendizaje. Persistir la orden con Add y SaveChangesAsync.

Procedimiento guiado.

1. Abre el POST creado en 2.2.5; localiza `new Orden { Cliente = ..., Material = ..., Cantidad = ... }` y comprueba que el servidor asigna automáticamente Id, estado y fecha.
2. Coloca un punto de interrupción en `db.Ordenes.Add(orden)` y otro en `await db.SaveChangesAsync()`. Arranca con F5.
3. Envía desde `AceriaApp.http` un POST válido de prueba; observa que `Add` incorpora la entidad al contexto y que `SaveChangesAsync` confirma la operación.
4. Consulta la tabla con el Explorador de objetos de SQL Server o mediante el GET de colección y comprueba que la orden quedó almacenada. No borres `SaveChangesAsync` de la versión final.

Referencia de código: `src/AceriaApp.Api/Program.cs`. Busca el método HTTP correspondiente y su llamada a `SaveChangesAsync`. Comprueba el comportamiento indicado en la acción antes de continuar.

Por qué funciona así. El POST crea un recurso y debe confirmar la inserción mediante SaveChangesAsync. CreatedAtRoute devuelve 201 y un enlace Location utilizable mediante GET.

Verificación concreta. El Id generado es un GUID y el nuevo registro se recupera tras una petición HTTP posterior; no basta con que se haya ejecutado `Add` sin excepción.

Si algo falla. Si el POST no persiste, verifica SaveChangesAsync; si Location falta, revisa CreatedAtRoute y el nombre de GET.

PASO 2.2.7 — Devolver 201 y Location con CreatedAtRoute

Resultado de aprendizaje. Devolver 201 y Location con CreatedAtRoute.

Procedimiento guiado.

1. En el mismo POST localiza `Results.CreatedAtRoute("ObtenerOrdenPorId", new { id = orden.Id }, OrdenDto.Desde(orden))`. No lo añadas otra vez: ya aparece en el handler completo de 2.2.5.
2. Busca en el GET individual el `.WithName("ObtenerOrdenPorId");` comprueba que ese nombre coincide exactamente con el utilizado por `CreatedAtRoute`.
3. Ejecuta el POST válido desde el cliente HTTP e inspecciona **código 201**, cabecera **Location** y cuerpo con el Id de la nueva orden.
4. Copia la URL de `Location` y ejecuta inmediatamente GET. Si falta la cabecera, revisa el nombre de ruta; si GET responde 404, comprueba el Id realmente guardado.

Archivo de trabajo: **`src/AceriaApp.Api/Program.cs`**.

```
// Este retorno ya está en el único MapPost creado en 2.2.5.
// PASO 2.2.7: comprueba que el GET tiene el nombre ObtenerOrdenPorId.
return Results.CreatedAtRoute(
    "ObtenerOrdenPorId",
    new { id = orden.Id },
    OrdenDto.Desde(orden));
```

Por qué funciona así. El POST crea un recurso y debe confirmar la inserción mediante SaveChangesAsync. CreatedAtRoute devuelve 201 y un enlace Location utilizable mediante GET.

Verificación concreta. POST 201 + `Location` utilizable y GET posterior 200 con el mismo Id. El servidor no debe limitarse a devolver 200 sin recurso creado.

Si algo falla. Si el POST no persiste, verifica SaveChangesAsync; si Location falta, revisa CreatedAtRoute y el nombre de GET.

PASO 2.2.8 — Enviar POST y consultar por GET la orden creada

Resultado de aprendizaje. Enviar POST y consultar por GET la orden creada.

Procedimiento guiado.

1. En `AceriaApp.http`, prepara un POST con `Content-Type: application/json` y el cuerpo `{ "cliente": "ACX-001", "material": "CHAPA-3MM", "cantidad": 10 }`.
2. Arranca la API con LocalDB, ejecuta la solicitud y conserva el Id y `Location` devueltos.
3. Sustituye `{id}` de una petición GET por el GUID real y ejecútala; verifica que los tres campos son iguales a los enviados y que el estado inicial es `Pendiente`.
4. Anota qué resultados demuestran el contrato (201 y 200) y cuáles demuestran persistencia (el recurso sigue existiendo en un segundo request).

Referencia de código: `src/AceriaApp.Api/Program.cs`. Busca el método HTTP correspondiente y su llamada a `SaveChangesAsync`. Comprueba el comportamiento indicado en la acción antes de continuar.

Por qué funciona así. El POST crea un recurso y debe confirmar la inserción mediante SaveChangesAsync. CreatedAtRoute devuelve 201 y un enlace Location utilizable mediante GET.

Verificación concreta. La respuesta GET individual debe mostrar el mismo `id`, `cliente`, `material`, `cantidad=10` y `estado=Pendiente`.

Si algo falla. Si el POST no persiste, verifica SaveChangesAsync; si Location falta, revisa CreatedAtRoute y el nombre de GET.

PASO 2.2.9 — Rechazar provisionalmente peticiones sin datos

Resultado de aprendizaje. Rechazar provisionalmente peticiones sin datos.

Procedimiento guiado.

1. Abre las primeras condiciones del POST de 2.2.5 y localiza las comprobaciones provisionales de `Cliente`, `Material` y `Cantidad <= 0`.
2. Prepara tres solicitudes, cambiando una propiedad cada vez: `cliente` vacío, `material` vacío y `cantidad: 0`; conserva las otras dos propiedades correctas.
3. Ejecuta cada POST y verifica el código **400 Bad Request** y el cuerpo de error simple que devuelve esta etapa; comprueba con GET que no se han añadido órdenes inválidas.
4. No incorpores todavía FluentValidation ni dupliques el `if`. La sustitución por validadores y filtros se realizará expresamente en 2.3.

Por qué funciona así. Un cuerpo incorrecto es distinto de una regla del negocio. En 2.2 el 400 es provisional; 501 unidades reciben 422 por la capacidad limitada del taller.

Verificación concreta. Tres POST inválidos devuelven 400 y no crean registros. El cuerpo 400 todavía no tiene por qué ser `ValidationProblemDetails`.

Si algo falla. Si un 501 da 400, la condición provisional está mezclada con la política de capacidad; distingue ambos casos.

PASO 2.2.10 — Comprobar la política del taller y respuesta 422

Resultado de aprendizaje. Comprobar la política del taller y respuesta 422.

Procedimiento guiado.

1. Prepara un POST con `Cliente` y `Material` válidos y `cantidad: 501`; no lo mezcles con un campo vacío que activaría la regla provisional de 400.
2. Coloca un punto de interrupción en `if (request.Cantidad > 500)` del POST y ejecuta la solicitud desde el archivo HTTP.
3. Comprueba que se alcanza `Results.Problem(... statusCode: 422)` y que no se ejecuta `db.Ordenes.Add`.
4. Anota la diferencia entre un valor sintácticamente correcto pero no aceptable por la capacidad del taller y una entrada obligatoria ausente. Repite con cantidad 500 para observar que no entra en esta condición.

Por qué funciona así. Un cuerpo incorrecto es distinto de una regla del negocio. En 2.2 el 400 es provisional; 501 unidades reciben 422 por la capacidad limitada del taller.

Verificación concreta. Cantidad 501 produce HTTP 422 sin inserción; 500 pasa el límite de negocio y puede crear una orden si las demás condiciones son válidas.

Si algo falla. Si un 501 da 400, la condición provisional está mezclada con la política de capacidad; distingue ambos casos.

PASO 2.2.11 — Programar MapPut con identificador Guid

Resultado de aprendizaje. Programar MapPut con identificador Guid.

Procedimiento guiado.

1. Debajo del POST y antes de `app.Run()`, pega el único `MapPut("/api/v1/ordenes/{id:guid}", ...)` del ejemplo, con `Guid id`, `CrearOrdenRequest` y DbContext.
2. Comprueba el flujo dentro del manejador: 400 provisional, búsqueda por GUID, 404 si no existe, 409 si está finalizada, 422 si supera capacidad, actualización y `SaveChangesAsync`.
3. Deja el mismo `.WithName("ReemplazarOrden")` y `.WithTags("Ordenes")`, sin cambiar el nombre del GET anterior.
4. Guarda y compila; la modificación de los campos y su prueba se explican en 2.2.12–2.2.13, no mediante la creación de un segundo PUT.

Archivo de trabajo: **`src/AceriaApp.Api/Program.cs`**.

```
// INICIO PASO 2.2.11: sustitución completa de los campos editables.
app.MapPut("/api/v1/ordenes/{id:guid}",async (Guid id,CrearOrdenRequest request,AceriaDbContext db)=>
{
    if(string.IsNullOrWhiteSpace(request.Cliente)||string.IsNullOrWhiteSpace(request.Material)||
request.Cantidad<=0)
        return Results.BadRequest(new {error="Cliente, material y cantidad son obligatorios."});
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"No se puede modificar una orden finalizada.",statusCode:409);
    if(request.Cantidad>500) return Results.Problem(
        title:"El taller solo admite lotes de hasta 500.",statusCode:422);
    orden.Cliente=request.Cliente;orden.Material=request.Material;
```

```
    orden.Cantidad=request.Cantidad;
    await db.SaveChangesAsync();
    return Results.NoContent();
})
.WithName("ReemplazarOrden").WithTags("Ordenes")


;
```

Por qué funciona así. PUT sustituye los campos editables de una orden existente. Si el Guid no identifica un recurso debe devolver 404, no crear silenciosamente otro.

Verificación concreta. El proyecto compila con un PUT y su ruta individual; el GET y POST anteriores siguen disponibles.

Si algo falla. Si el PUT no actualiza, revisa FindAsync(id) y SaveChangesAsync. El GUID no debe cambiar.

PASO 2.2.12 — Modificar Cliente, Material y Cantidad con PUT

Resultado de aprendizaje. Modificar Cliente, Material y Cantidad con PUT.

Procedimiento guiado.

1. Crea una orden `Pendiente` mediante POST y conserva su GUID. No utilices una orden previamente finalizada.
2. En el `MapPut` de 2.2.11, examina `FindAsync(id)`, las tres asignaciones `Cliente`, `Material`, `Cantidad` y la llamada `SaveChangesAsync()`.
3. Envía PUT al GUID real con los valores `ACX-002`, `CHAPA-5MM` y `14`; no incluyas en el JSON `Id`, `Estado` ni `FechaCreacion`.
4. Con un punto de interrupción comprueba que el contexto encuentra la misma entidad, que el Id no cambia y que se sustituyen los tres campos editables.

Referencia de código: `src/AceriaApp.Api/Program.cs`. Busca el método HTTP correspondiente y su llamada a `SaveChangesAsync`. Comprueba el comportamiento indicado en la acción antes de continuar.

Por qué funciona así. PUT sustituye los campos editables de una orden existente. Si el Guid no identifica un recurso debe devolver 404, no crear silenciosamente otro.

Verificación concreta. PUT del recurso pendiente devuelve 204 y conserva el mismo GUID; la actualización debe afectar Cliente, Material y Cantidad, sin crear una nueva fila.

Si algo falla. Si el PUT no actualiza, revisa FindAsync(id) y SaveChangesAsync. El GUID no debe cambiar.

Caso práctico completo — Una transacción observable: POST, Location, GET y PUT

El primer recorrido de escritura empieza con el método POST. Ejecuta la aplicación y abre `AceriaApp.http`. Envía un cuerpo JSON válido:

```
{"cliente":"ACX-001","material":"CHAPA-3MM","cantidad":10}
```

Utiliza la URI `/api/v1/ordenes` y el verbo POST. La operación construye una entidad nueva de Orden con los datos del contrato. El identificador y la fecha se asignan en el servidor. Cuando `SaveChangesAsync` confirma la inserción, `Results.CreatedAtRoute` prepara un estado 201 con una cabecera `Location`. Abre esa cabecera: no es un mensaje decorativo, sino una ruta que puede usarse después para recuperar la orden.

Envía ahora GET a la URI de `Location` y comprueba que el JSON contiene cliente ACX-001, material CHAPA-3MM y cantidad 10. Guarda el Guid obtenido: lo necesitarás para PUT, PATCH y DELETE. Si GET no encuentra la orden, pregunta qué base de datos usa cada petición y si el POST realizó

`SaveChangesAsync`. El hecho de que el POST devuelva un cuerpo con un ID no demuestra por sí solo que se haya insertado una fila.

Para entender PUT, envía a `/api/v1/ordenes/{id}` un cuerpo con cliente ACX-002, material CHAPA-5MM y cantidad 14. El endpoint localiza la orden mediante `FindAsync(id)`, modifica tres propiedades editables y persiste el cambio. El resultado normal es 204, sin un cuerpo de respuesta que copiar. Ejecuta un nuevo GET: el Guid debe ser el mismo, mientras que los tres campos editables reflejan los nuevos valores.

Comprobación: compara lo que cambió y lo que permaneció intacto. El ID, el Estado y la FechaCreacion son valores controlados por el servidor. Si un PUT crea silenciosamente una orden con un Guid inexistente, no está cumpliendo el contrato concreto de este ejemplo.

PASO 2.2.13 — Comprobar 204 y consultar por GET los cambios

Resultado de aprendizaje. Comprobar 204 y consultar por GET los cambios.

Procedimiento guiado.

1. Después de ejecutar PUT, comprueba que la respuesta es **204 No Content**: un cuerpo vacío es correcto para este resultado.
2. Envía un GET al mismo GUID usando la petición de detalle de tu archivo `.http`.
3. Compara cada campo con los valores anteriores: Cliente, Material y Cantidad deben haberse sustituido; Estado debe seguir siendo `Pendiente`.
4. Repite un PUT idéntico y comprueba que no aparece otra orden. Registra la evidencia del efecto persistente en lugar de deducirlo solo del 204.

Referencia de código: `src/AceriaApp.Api/Program.cs`. Busca el método HTTP correspondiente y su llamada a `SaveChangesAsync`. Comprueba el comportamiento indicado en la acción antes de continuar.

Por qué funciona así. PUT sustituye los campos editables de una orden existente. Si el Guid no identifica un recurso debe devolver 404, no crear silenciosamente otro.

Verificación concreta. Dos PUT idénticos conservan una sola orden con los nuevos valores. GET posterior 200; PUT 204 sin JSON de respuesta.

Si algo falla. Si el PUT no actualiza, revisa FindAsync(id) y SaveChangesAsync. El GUID no debe cambiar.

PASO 2.2.14 — Programar MapPatch para cambiar solo Cantidad

Resultado de aprendizaje. Programar MapPatch para cambiar solo Cantidad.

Procedimiento guiado.

1. En `Program.cs`, incorpora el único `MapPatch("/api/v1/ordenes/{id:guid}/cantidad", ...)` del bloque C#.
2. Comprueba que el contrato utilizado es **`ActualizarCantidadRequest`**, no `CrearOrdenRequest`, y que la ruta incluye el segmento final `/cantidad`.
3. Sigue las ramas de búsqueda, rechazo por estado y capacidad, y asignación exclusiva de `orden.Cantidad`; guarda con `SaveChangesAsync`.
4. Compila el proyecto. Prepara en `AceriaApp.http` un PATCH con `Content-Type: application/json` y cuerpo `{ "cantidad": 15 }`.

Archivo de trabajo: **`src/AceriaApp.Api/Program.cs`**.

```
// INICIO PASO 2.2.14: cambiar SOLO la cantidad.
app.MapPatch("/api/v1/ordenes/{id:guid}/cantidad",
    async (Guid id,ActualizarCantidadRequest request,AceriaDbContext db)=>
{
```

```
    if(request.Cantidad<=0) return Results.BadRequest(new {error="Cantidad debe ser positiva."});
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"No se puede modificar una orden finalizada.",statusCode:409);
    if(request.Cantidad>500) return Results.Problem(
        title:"El taller solo admite lotes de hasta 500.",statusCode:422);
    orden.Cantidad=request.Cantidad;
    await db.SaveChangesAsync();
    return Results.Ok(OrdenDto.Desde(orden));
})
.WithName("CambiarCantidad").WithTags("Ordenes")


;
```

Por qué funciona así. PATCH realiza un cambio parcial de Cantidad y conserva los demás datos. Su resultado debe verificarse con una lectura posterior.

Verificación concreta. Hay un solo handler PATCH de cantidad, con cuerpo tipado, respuesta 200 en operación válida y sin necesidad de JSON Patch genérico.

Si algo falla. Si PATCH cambia otros campos, no reasignes Cliente ni Material desde ActualizarCantidadRequest.

PASO 2.2.15 — Comprobar que PATCH no modifica los demás campos

Resultado de aprendizaje. Comprobar que PATCH no modifica los demás campos.

Procedimiento guiado.

1. Antes de aplicar PATCH, ejecuta un GET del recurso y anota los valores actuales de Cliente, Material y Cantidad.
2. Envía PATCH con una cantidad diferente y comprueba su respuesta **200** con el DTO actualizado.
3. Repite GET a la misma URI de detalle y compara los campos: Cantidad habrá cambiado y Cliente/Material deben permanecer idénticos.
4. Si se modificó algún otro campo, revisa el handler de 2.2.14; no añadas una segunda ruta PATCH para corregirlo.

Referencia de código: `src/AceriaApp.Api/Program.cs`. Busca el método HTTP correspondiente y su llamada a `SaveChangesAsync`. Comprueba el comportamiento indicado en la acción antes de continuar.

Por qué funciona así. PATCH realiza un cambio parcial de Cantidad y conserva los demás datos. Su resultado debe verificarse con una lectura posterior.

Verificación concreta. PATCH 200 y GET posterior 200; solo cambia `Cantidad`. El estado y el Id siguen siendo los mismos.

Si algo falla. Si PATCH cambia otros campos, no reasignes Cliente ni Material desde ActualizarCantidadRequest.

PASO 2.2.16 — Programar MapDelete para eliminar una orden

Resultado de aprendizaje. Programar MapDelete para eliminar una orden.

Procedimiento guiado.

1. Debajo del PATCH, agrega el `MapDelete("/api/v1/ordenes/{id:guid}", ...)` del bloque de este paso.
2. Verifica que primero consulta `FindAsync(id)`, después descarta recursos inexistentes (404) o finalizados (409), y solo entonces ejecuta `Remove` y `SaveChangesAsync`.

3. Prepara una orden nueva `Pendiente` para probar el borrado, evitando utilizar la orden finalizada de otros ejercicios.
4. Compila y ejecuta DELETE contra el GUID real. Comprueba que la respuesta es **204 No Content**.

Archivo de trabajo: **`src/AceriaApp.Api/Program.cs`**.

```
// INICIO PASO 2.2.16: eliminar recurso persistido.
app.MapDelete("/api/v1/ordenes/{id:guid}",async (Guid id,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"No se puede eliminar una orden finalizada.",statusCode:409);
    db.Ordenes.Remove(orden);
    await db.SaveChangesAsync();
    return Results.NoContent();
})
.WithName("EliminarOrden").WithTags("Ordenes")

;
```

Por qué funciona así. DELETE quita un recurso persistido. La consulta posterior puede devolver 404; esa diferencia no impide que la eliminación sea idempotente.

Verificación concreta. El DELETE de una orden pendiente finaliza con 204 y no deja esa orden en la tabla; el código contiene un solo MapDelete.

Si algo falla. Si DELETE devuelve éxito pero GET encuentra la orden, revisa Remove y SaveChangesAsync.

PASO 2.2.17 — Verificar 204 seguido de GET 404

Resultado de aprendizaje. Verificar 204 seguido de GET 404.

Procedimiento guiado.

1. Guarda el GUID de la orden que acabas de eliminar y ejecuta GET sobre su URI individual.
2. Comprueba **404** porque la orden ya no existe. Repite DELETE sobre el mismo GUID y comprueba también la respuesta de recurso inexistente.
3. Comprueba con GET de colección que no se ha recuperado la orden ni creado otra en su lugar.
4. Escribe una conclusión breve: repetir DELETE puede producir distintos códigos HTTP, pero no modifica el estado final una vez eliminado el recurso.

Referencia de código: `src/AceriaApp.Api/Program.cs`. Busca el método HTTP correspondiente y su llamada a `SaveChangesAsync`. Comprueba el comportamiento indicado en la acción antes de continuar.

Por qué funciona así. DELETE quita un recurso persistido. La consulta posterior puede devolver 404; esa diferencia no impide que la eliminación sea idempotente.

Verificación concreta. Secuencia verificable: primer DELETE 204, GET 404 y segundo DELETE 404, con ausencia persistente de la orden.

Si algo falla. Si DELETE devuelve éxito pero GET encuentra la orden, revisa Remove y SaveChangesAsync.

PASO 2.2.18 — Programar la transición de estado finalizar

Resultado de aprendizaje. Programar la transición de estado finalizar.

Procedimiento guiado.

1. Registra un nuevo endpoint `POST /api/v1/ordenes/{id:guid}/finalizar` copiando el bloque del laboratorio **antes de `app.Run()`**; no copies `app.Run()` ni otra clase `Program`.
2. Dentro de la lambda localiza `FindAsync`, las respuestas 404/409, la asignación `Estado = "Finalizada"` y `SaveChangesAsync`.
3. Crea una orden nueva pendiente, anota su Id y ejecuta POST a la URI `/finalizar` correspondiente, sin cuerpo JSON si no lo necesita.
4. Repite POST finalizar con el mismo GUID y observa 409; explica por qué terminar dos veces la misma transición no se acepta como una segunda operación válida.

Archivo de trabajo: **`src/AceriaApp.Api/Program.cs`**.

```
// INICIO PASO 2.2.18: transición explícita del dominio.
app.MapPost("/api/v1/ordenes/{id:guid}/finalizar",async (Guid id,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"La orden ya estaba finalizada.",statusCode:409);
    orden.Estado="Finalizada";
    await db.SaveChangesAsync();
    return Results.Ok(OrdenDto.Desde(orden));
})
.WithName("FinalizarOrden").WithTags("Ordenes")

;
```

Por qué funciona así. La transición Finalizada cambia el estado de negocio. PUT, PATCH y DELETE deben respetarlo y producir 409 si se intenta una operación incompatible.

Verificación concreta. Primer finalizar 200 con DTO de estado `Finalizada`; segundo finalizar 409, y el GET de detalle conserva el estado final.

Si algo falla. Si una orden finalizada cambia de estado o desaparece, comprueba la condición Estado antes de escribir.

PASO 2.2.19 — Bloquear PUT y PATCH para órdenes finalizadas

Resultado de aprendizaje. Bloquear PUT y PATCH para órdenes finalizadas.

Procedimiento guiado.

1. Selecciona una orden cuyo estado sea `Finalizada` y conserva su Id; crea otra `Pendiente` para disponer de control positivo.
2. Ejecuta PUT con datos válidos contra la orden finalizada y comprueba **409 Conflict**. Haz después PATCH a `/cantidad` sobre el mismo GUID.
3. Ejecuta GET después de cada intento y verifica que ninguno modificó los campos persistidos.
4. Abre los handlers PUT y PATCH para localizar sus comprobaciones de `Estado == "Finalizada"`, ya incluidas en 2.2.11 y 2.2.14. No las pegues otra vez.

Referencia de código: `src/AceriaApp.Api/Program.cs`. Busca el método HTTP correspondiente y su llamada a `SaveChangesAsync`. Comprueba el comportamiento indicado en la acción antes de continuar.

Por qué funciona así. La transición Finalizada cambia el estado de negocio. PUT, PATCH y DELETE deben respetarlo y producir 409 si se intenta una operación incompatible.

Verificación concreta. PUT y PATCH de una orden finalizada producen 409 y dejan la entidad inalterada. Una orden pendiente sigue siendo modificable.

Si algo falla. Si una orden finalizada cambia de estado o desaparece, comprueba la condición Estado antes de escribir.

PASO 2.2.20 — Bloquear DELETE y devolver conflicto 409

Resultado de aprendizaje. Bloquear DELETE y devolver conflicto 409.

Procedimiento guiado.

1. Conserva el GUID finalizado del ejercicio anterior y ejecuta DELETE sobre él.
2. Comprueba que devuelve **409** y que GET de detalle sigue devolviendo **200** con Estado `Finalizada`.
3. Contrasta el comportamiento con un DELETE de una orden pendiente (204) y con un GUID inexistente (404).
4. Señala en `MapDelete` el `if (orden.Estado == "Finalizada")` que impide llegar a `db.Ordenes.Remove`; no crees otro endpoint.

Referencia de código: `src/AceriaApp.Api/Program.cs`. Busca el método HTTP correspondiente y su llamada a `SaveChangesAsync`. Comprueba el comportamiento indicado en la acción antes de continuar.

Por qué funciona así. La transición Finalizada cambia el estado de negocio. PUT, PATCH y DELETE deben respetarlo y producir 409 si se intenta una operación incompatible.

Verificación concreta. El servicio diferencia 204, 404 y 409 según exista el recurso y esté permitido eliminarlo; 409 no altera datos.

Si algo falla. Si una orden finalizada cambia de estado o desaparece, comprueba la condición Estado antes de escribir.

PASO 2.2.21 — Escribir pruebas HTTP POST y Location

Resultado de aprendizaje. Escribir pruebas HTTP POST y Location.

Procedimiento guiado.

1. En el proyecto xUnit heredado de 2.1 crea `EscrituraTests.cs` y pega la clase y el primer método del bloque `CrearYConsultar_Es201Y200`.
2. Conserva imports `System.Net`, `System.Net.Http.Json`, `AceriaApp.Api.Contratos`, `AceriaApp.Api.Modelo` y `Xunit`. No declares otra clase `EscrituraTests`.
3. Recorre el test: crea `ApiFactory`, ejecuta POST, comprueba 201 y `Location`, y solicita GET mediante la cabecera recibida.
4. Guarda y ejecuta el proyecto de tests. Los tres GET existentes y este nuevo caso deben superar sus aserciones antes de ampliar la clase.

Archivo de trabajo: **`tests/AceriaApp.Api.Tests/EscrituraTests.cs`**.

```
using System.Net;
using System.Net.Http.Json;
using AceriaApp.Api.Contratos;
using AceriaApp.Api.Modelo;
using Xunit;

namespace AceriaApp.Api.Tests;

public class EscrituraTests
{
    // PASO 2.2.21: primera prueba de escritura.
    [Fact]
    public async Task CrearYConsultar_Es201Y200()
    {
        using var f = new ApiFactory();
        using var client = f.CreateClient();
        var created = await client.PostAsJsonAsync("/api/v1/ordenes",
```

```
            new CrearOrdenRequest("ACX", "CHAPA", 10));
        Assert.Equal(HttpStatusCode.Created, created.StatusCode);
        Assert.NotNull(created.Headers.Location);
        var get = await client.GetAsync(created.Headers.Location);
        Assert.Equal(HttpStatusCode.OK, get.StatusCode);
    }
}
```

Por qué funciona así. Los tests deben confirmar los efectos de POST, PUT, PATCH, DELETE, la capacidad del taller y el conflicto de estado a través de peticiones HTTP reales.

Verificación concreta. La nueva prueba demuestra POST→Location→GET y no solo POST 201. La clase se ampliará en los pasos siguientes.

Si algo falla. Si una prueba de escritura pasa sin volver a consultar la orden, no ha demostrado persistencia; añade el GET posterior.

PASO 2.2.22 — Escribir pruebas HTTP PUT, PATCH y DELETE

Resultado de aprendizaje. Escribir pruebas HTTP PUT, PATCH y DELETE.

Procedimiento guiado.

1. Abre `EscrituraTests.cs` y sitúa el cursor **dentro de su clase**, antes de la última llave `}`.
2. Inserta únicamente el nuevo método `EditarYEliminar_CambiaPersistencia` del bloque de este paso; conserva `CrearYConsultar_Es201Y200`.
3. Sigue el recorrido del caso: POST crea, PUT actualiza, PATCH cambia cantidad, DELETE elimina y GET confirma el 404 final.
4. Ejecuta xUnit y, si falla, identifica cuál de las aserciones indica el primer contrato incumplido. No dupliques la clase ni vuelvas a crear el archivo.

Archivo de trabajo: `tests/AceriaApp.Api.Tests/EscrituraTests.cs`.

```
// Añade dentro de EscrituraTests, sin crear otra clase:
[Fact]
public async Task EditarYEliminar_CambiaPersistencia()
{
    using var f = new ApiFactory();
    using var c = f.CreateClient();
    var made = await c.PostAsJsonAsync("/api/v1/ordenes",
        new CrearOrdenRequest("ACX", "CHAPA", 10));
    var dto = await made.Content.ReadFromJsonAsync<OrdenDto>();
    Assert.NotNull(dto);
    var put = await c.PutAsJsonAsync("/api/v1/ordenes/" + dto.Id,
        new CrearOrdenRequest("ACX2", "CHAPA", 11));
    Assert.Equal(HttpStatusCode.NoContent, put.StatusCode);
    var patch = await c.PatchAsJsonAsync(
        "/api/v1/ordenes/" + dto.Id + "/cantidad",
        new ActualizarCantidadRequest(12));
    Assert.Equal(HttpStatusCode.OK, patch.StatusCode);
    var changed = await patch.Content.ReadFromJsonAsync<OrdenDto>();
    Assert.Equal(12, changed?.Cantidad);
    var del = await c.DeleteAsync("/api/v1/ordenes/" + dto.Id);
    Assert.Equal(HttpStatusCode.NoContent, del.StatusCode);
    var gone = await c.GetAsync("/api/v1/ordenes/" + dto.Id);
    Assert.Equal(HttpStatusCode.NotFound, gone.StatusCode);
}
```

Por qué funciona así. Los tests deben confirmar los efectos de POST, PUT, PATCH, DELETE, la capacidad del taller y el conflicto de estado a través de peticiones HTTP reales.

Verificación concreta. La suite incluye los tres GET heredados y los dos casos de escritura nuevos, sin pruebas de plantilla ni clases duplicadas.

Si algo falla. Si una prueba de escritura pasa sin volver a consultar la orden, no ha demostrado persistencia; añade el GET posterior.

PASO 2.2.23 — Probar automáticamente 422 y 409

Resultado de aprendizaje. Probar automáticamente 422 y 409.

Procedimiento guiado.

1. Dentro de la misma clase `EscrituraTests`, añade los dos métodos del fragmento de este paso: `ReglaCapacidad_Es422` y `OrdenFinalizada_NoSeEdita409`.
2. En el primer test prepara cantidad 501 y comprueba 422; en el segundo, crea una orden, finalízala y verifica 409 al intentar PUT.
3. Ejecuta `dotnet test tests/AceriaApp.Api.Tests/AceriaApp.Api.Tests.csproj` desde la raíz de esta etapa o usa **Test > Run All Tests** en Visual Studio.
4. Verifica el recuento **7 superados, 0 fallidos**, con todos los tests del punto 2.1 preservados.

Añade estas dos pruebas dentro de `EscrituraTests` (sin duplicar la clase):

```
[Fact]
public async Task ReglaCapacidad_Es422()
{
    using var f = new ApiFactory();
    var r = await f.CreateClient().PostAsJsonAsync("/api/v1/ordenes",
        new CrearOrdenRequest("ACX", "CHAPA", 501));
    Assert.Equal(HttpStatusCode.UnprocessableEntity, r.StatusCode);
}

[Fact]
public async Task OrdenFinalizada_NoSeEdita409()
{
    using var f = new ApiFactory();
    using var c = f.CreateClient();
    var made = await c.PostAsJsonAsync("/api/v1/ordenes",
        new CrearOrdenRequest("ACX", "CHAPA", 2));
    var dto = await made.Content.ReadFromJsonAsync<OrdenDto>();
    Assert.NotNull(dto);
    var done = await c.PostAsync(
        "/api/v1/ordenes/" + dto.Id + "/finalizar", null);
    Assert.Equal(HttpStatusCode.OK, done.StatusCode);
    var put = await c.PutAsJsonAsync("/api/v1/ordenes/" + dto.Id,
        new CrearOrdenRequest("ACX", "CHAPA", 3));
    Assert.Equal(HttpStatusCode.Conflict, put.StatusCode);
}
```

Por qué funciona así. Los tests deben confirmar los efectos de POST, PUT, PATCH, DELETE, la capacidad del taller y el conflicto de estado a través de peticiones HTTP reales.

Verificación concreta. Se superan 7 pruebas acumuladas: 3 GET + 4 de escritura. El error 422 y el conflicto 409 están comprobados mediante HTTP.

Si algo falla. Si una prueba de escritura pasa sin volver a consultar la orden, no ha demostrado persistencia; añade el GET posterior.

PASO 2.2.24 — Reto: comprobar idempotencia y efecto persistente

Resultado de aprendizaje. Reto: comprobar idempotencia y efecto persistente.

Procedimiento guiado.

1. Reto opcional: trabaja en una copia aparte para no alterar la etapa 2.2 de referencia.
2. Crea una orden nueva, envía dos PUT **idénticos** al mismo GUID y después ejecuta GET; anota que solo queda un recurso con el estado esperado.

3. Crea otra orden pendiente, ejecuta DELETE dos veces y luego GET; registra 204, 404 y 404 como secuencia posible.
4. Razona qué significa idempotencia a partir del estado final, no de que los códigos HTTP de dos solicitudes consecutivas deban ser iguales.

Por qué funciona así. La idempotencia habla del efecto final: repetir PUT no multiplica los recursos, y repetir DELETE no restaura lo eliminado. Las respuestas HTTP pueden diferir.

Verificación concreta. La evidencia del reto muestra una única entidad tras repetir PUT y ausencia de entidad tras repetir DELETE; no afecta a las 7 pruebas base.

Si algo falla. Si se crean órdenes duplicadas al repetir PUT, se está usando Add en lugar de actualizar la entidad encontrada.

Caso práctico completo — PATCH, eliminación e idempotencia: observar el estado final

Conserva la orden creada en el caso anterior. Ejecuta PATCH sobre `/api/v1/ordenes/{id}/cantidad` con el cuerpo:

```
{"cantidad":15}
```

Comprueba que el código es 200 y que la representación devuelta contiene Cantidad=15, Cliente=ACX-002 y Material=CHAPA-5MM. Esto demuestra que PATCH no ha reconstruido el objeto perdiendo otras propiedades. En el método C# correspondiente solo se reasigna `orden.Cantidad`; no se realiza un `new Orden` ni un `db.Ordenes.Add`.

A continuación ejecuta DELETE con el mismo Guid. El servidor localiza la orden, ejecuta `Remove`, confirma `SaveChangesAsync` y devuelve 204. Envía después GET del mismo identificador: debe producir 404. El recurso ya no existe, aunque la ruta como plantilla siga registrada en la API. Si vuelves a ejecutar DELETE, puede responder 404 porque la orden ya no se encuentra. Esto no contradice la idempotencia de DELETE: el estado final sigue siendo ausencia del recurso.

Realiza una segunda secuencia para reproducir conflicto de estado. Crea otra orden, ejecuta POST sobre `/{id}/finalizar` y después intenta actualizarla con PUT o PATCH. La operación debe devolver 409 sin modificar los datos. La política del taller comprueba Estado=Finalizada antes de escribir; el problema no es que el JSON sea inválido, sino que esa transición ya no está permitida. En otra petición distinta envía Cantidad=501: aunque el rango de valores del contrato admite 501, la capacidad del taller está limitada a 500 y devuelve 422.

Reto final: repite exactamente el mismo PUT sobre una orden pendiente dos veces. Comprueba que el cliente, material y cantidad quedan iguales. La idempotencia se entiende observando el efecto final, no contando solamente códigos 204.

Matriz de comprobación del punto

Comportamiento	Resultado esperado
`POST /api/v1/ordenes`	201 + Location
`PUT /{id}`	204
`PATCH /{id}/cantidad`	200
`DELETE /{id}`	204 / 404
`POST /{id}/finalizar`	200 / 409
`Cantidad 501`	422

Reto integrador. Reconstruye sin consultar la solución el último endpoint o validación practicada y añade una prueba HTTP que demuestre su resultado. Describe el recurso, el verbo, el contrato y el código devuelto. Si no puedes reproducir una respuesta, vuelve al primer paso que introdujo el comportamiento.

Punto 2.3 — Validación efectiva de peticiones con FluentValidation

Proyecto comprobado: `PROYECTO/2.3-validacion-peticiones`. El punto comienza estudiando el contrato de recursos y concluye con una ejecución verificable. Ninguna comprobación sustituye a un endpoint realmente implementado.

PASO 2.3.1 — Comprobar endpoints anteriores y localizar sus contratos

Resultado de aprendizaje. Comprobar endpoints anteriores y localizar sus contratos.

Procedimiento guiado.

1. Crea tu carpeta de trabajo de 2.3 copiando el **estado final de tu etapa 2.2** (proyecto API, tests y `.http`). No copies el proyecto de referencia 2.3 ya terminado.
2. Abre la nueva copia en Visual Studio y ejecuta los tests heredados; verifica que los GET y las operaciones de escritura siguen respondiendo correctamente.
3. Prepara un GUID de una orden pendiente y otro de una orden finalizada para utilizarlos como casos de prueba de validación y conflicto.
4. Anota qué 400 se obtienen hoy mediante `if` provisionales de `Program.cs`. Estos controles se reemplazarán por validadores y filtros sin cambiar las rutas públicas.

Por qué funciona así. La deserialización de un cuerpo JSON y la validación semántica no son lo mismo. Primero el framework interpreta el mensaje y después se comprueban las reglas de datos.

Verificación concreta. Antes de modificar el código deben pasar las 7 pruebas de 2.2. Has identificado los `if` provisionales y conservas una copia funcional.

Si algo falla. Si JSON roto produce un error inesperado, revisa Content-Type y la estructura del cuerpo antes del validador.

PASO 2.3.2 — Distinguir JSON incorrecto, semántica y negocio

Resultado de aprendizaje. Distinguir JSON incorrecto, semántica y negocio.

Procedimiento guiado.

1. En `AceriaApp.http` prepara tres cuerpos para POST: una cadena JSON sintácticamente rota, un JSON válido con `cantidad=0`, y otro con `cantidad=501` y los demás campos correctos.
2. Para cada caso predice **quién** debe rechazarlo: el enlazador JSON de ASP.NET Core, FluentValidation o la política de capacidad del manejador.
3. Ejecuta las tres solicitudes en el estado actual y guarda sus códigos y cuerpos. No supongas que todos los 400 usan idéntico formato.
4. Crea una tabla «entrada → nivel de validación → HTTP esperado → efecto sobre la tabla» y consérvala para repetir el experimento tras conectar los filtros.

Por qué funciona así. La deserialización de un cuerpo JSON y la validación semántica no son lo mismo. Primero el framework interpreta el mensaje y después se comprueban las reglas de datos.

Verificación concreta. La tabla distingue JSON mal formado, cantidad fuera de rango semántico y capacidad 501. Al completar 2.3 debe poder justificarse 400 frente a 422.

Si algo falla. Si JSON roto produce un error inesperado, revisa Content-Type y la estructura del cuerpo antes del validador.

PASO 2.3.3 — Instalar FluentValidation.DependencyInjectionExtensions

Resultado de aprendizaje. Instalar FluentValidation.DependencyInjectionExtensions.

Procedimiento guiado.

1. En el proyecto API abre **Administrar paquetes NuGet** y busca `FluentValidation.DependencyInjectionExtensions`. Selecciona la versión compatible con la solución de referencia (`11.11.0`).
2. Instálala **en `src/AceriaApp.Api`**, no solo en el proyecto xUnit; como alternativa ejecuta `dotnet add src/AceriaApp.Api/AceriaApp.Api.csproj package FluentValidation.DependencyInjectionExtensions --version 11.11.0`.
3. Abre el `.csproj` y comprueba la nueva dependencia. Ejecuta `dotnet restore` y revisa posibles conflictos de versiones.
4. No elimines `Microsoft.EntityFrameworkCore.SqlServer`. Instalar el paquete no registra validadores ni convierte automáticamente los POST en validaciones.

Por qué funciona así. La deserialización de un cuerpo JSON y la validación semántica no son lo mismo. Primero el framework interpreta el mensaje y después se comprueban las reglas de datos.

Verificación concreta. El proyecto API restaura dependencias y referencia FluentValidation. El comportamiento HTTP todavía es el de la etapa 2.2 hasta que conectes el filtro.

Si algo falla. Si JSON roto produce un error inesperado, revisa Content-Type y la estructura del cuerpo antes del validador.

PASO 2.3.4 — Crear CrearOrdenValidator y reglas de Cliente

Resultado de aprendizaje. Crear CrearOrdenValidator y reglas de Cliente.

Procedimiento guiado.

1. Crea la carpeta `Validacion` dentro de `src/AceriaApp.Api` y añade el archivo `Validadores.cs`.
2. Escribe el `using AceriaApp.Api.Contratos;`, el `using FluentValidation;` y el namespace `AceriaApp.Api.Validacion` del ejemplo.
3. Define `CrearOrdenValidator : AbstractValidator<CrearOrdenRequest>` y escribe dentro de su constructor la regla de `Cliente` con `NotEmpty().MaximumLength(100)`.
4. Guarda y compila; comprueba que `CrearOrdenRequest` proviene de la carpeta `Contratos` heredada. No copies todavía reglas de Material ni el validador de PATCH.

Archivo de trabajo: `src/AceriaApp.Api/Validacion/Validadores.cs`.

```
using AceriaApp.Api.Contratos;
using FluentValidation;

namespace AceriaApp.Api.Validacion;

public sealed class CrearOrdenValidator : AbstractValidator<CrearOrdenRequest>
{
    public CrearOrdenValidator()
    {
        RuleFor(x => x.Cliente).NotEmpty().MaximumLength(100);
    }
}
```

Por qué funciona así. FluentValidation expresa reglas sobre los contratos de entrada y devuelve errores por campo; no debe confundirse un record con el validador que lo inspecciona.

Verificación concreta. `Validadores.cs` compila con una primera clase de validación de Cliente. Las demás reglas se incorporarán a ese mismo archivo en pasos posteriores.

Si algo falla. Si RuleFor no compila, revisa FluentValidation y el using. Comprueba la diferencia entre NotEmpty y límites numéricos.

PASO 2.3.5 — Definir Material y rango válido de Cantidad

Resultado de aprendizaje. Definir Material y rango válido de Cantidad.

Procedimiento guiado.

1. Abre `Validacion/Validadores.cs` y entra en el constructor de `CrearOrdenValidator` que ya contiene la regla de Cliente.
2. Añade las dos líneas del fragmento del laboratorio: `Material` obligatorio con longitud máxima 100 y `Cantidad` en el rango inclusivo 1–10000.
3. Verifica el significado de los límites 0, 1, 500, 501, 10000 y 10001. El valor 501 pasa FluentValidation pero será rechazado por la regla del negocio.
4. Guarda y compila. Comprueba que las nuevas reglas se añadieron al constructor **existente**, no a un segundo validador con el mismo nombre.

Código de este paso. Añade las reglas dentro del mismo constructor, después de la de Cliente. El rango semántico `1..10000` es diferente del límite de capacidad de negocio `500` evaluado en el endpoint.

```
// Añade estas dos líneas al constructor de CrearOrdenValidator:
RuleFor(x => x.Material).NotEmpty().MaximumLength(100);
RuleFor(x => x.Cantidad).InclusiveBetween(1, 10000);
```

Por qué funciona así. FluentValidation expresa reglas sobre los contratos de entrada y devuelve errores por campo; no debe confundirse un record con el validador que lo inspecciona.

Verificación concreta. El validador acepta cantidad 1–10000 con campos correctos y rechaza 0, 10001 o campos vacíos. La capacidad 500 continúa siendo regla de negocio.

Si algo falla. Si RuleFor no compila, revisa FluentValidation y el using. Comprueba la diferencia entre NotEmpty y límites numéricos.

PASO 2.3.6 — Crear ActualizarCantidadValidator para PATCH

Resultado de aprendizaje. Crear ActualizarCantidadValidator para PATCH.

Procedimiento guiado.

1. En `Validadores.cs`, después de la llave final de `CrearOrdenValidator`, declara una **segunda clase** `ActualizarCantidadValidator : AbstractValidator<ActualizarCantidadRequest>`.
2. Escribe su constructor y agrega la regla `RuleFor(x => x.Cantidad).InclusiveBetween(1, 10000)` como indica el fragmento.
3. Comprueba que el tipo del validador corresponde a `ActualizarCantidadRequest` y no a `CrearOrdenRequest`, puesto que PATCH solo recibe una cantidad.
4. Guarda, compila e identifica en qué endpoints utilizarás cada tipo de validador al llegar a la conexión de filtros.

Archivo de trabajo: **`src/AceriaApp.Api/Validacion/Validadores.cs`**.

```
// Añade al final de Validadores.cs, fuera de CrearOrdenValidator:
public sealed class ActualizarCantidadValidator
    : AbstractValidator<ActualizarCantidadRequest>
{
    public ActualizarCantidadValidator()
```

```
    {
        RuleFor(x => x.Cantidad).InclusiveBetween(1, 10000);
    }
}
```

Por qué funciona así. FluentValidation expresa reglas sobre los contratos de entrada y devuelve errores por campo; no debe confundirse un record con el validador que lo inspecciona.

Verificación concreta. El mismo archivo contiene exactamente los dos validadores públicos esperados y ambos compilan sin referencias a DTOs de salida.

Si algo falla. Si RuleFor no compila, revisa FluentValidation y el using. Comprueba la diferencia entre NotEmpty y límites numéricos.

PASO 2.3.7 — Ejecutar validadores y estudiar ValidationResult

Resultado de aprendizaje. Ejecutar validadores y estudiar ValidationResult.

Procedimiento guiado.

1. Antes de conectar los filtros, crea **temporalmente** una prueba xUnit de exploración llamada `ValidadoresExploracionTests.cs` en el proyecto de tests.
2. En un método `[Fact]` instancia `new CrearOrdenValidator()` y llama a `Validate(new CrearOrdenRequest("", "CHAPA", 0))`; inspecciona `IsValid` y `Errors` con el depurador o aserciones xUnit.
3. Repite con `new CrearOrdenRequest("ACX", "CHAPA", 10)` y comprueba que cambia `IsValid`. Observa que un validador es una clase ejecutable aunque todavía no se invoque en HTTP.
4. **Elimina el archivo temporal de exploración** tras anotar el resultado: el estado canónico 2.3 incorpora pruebas de integración HTTP específicas en 2.3.22 y no debe conservar tests de laboratorio transitorios.

Por qué funciona así. FluentValidation expresa reglas sobre los contratos de entrada y devuelve errores por campo; no debe confundirse un record con el validador que lo inspecciona.

Verificación concreta. La exploración muestra un `ValidationResult` inválido con errores por Cliente y Cantidad y otro válido. Al terminar, la suite vuelve a contener solo los tests permanentes anteriores.

Si algo falla. Si RuleFor no compila, revisa FluentValidation y el using. Comprueba la diferencia entre NotEmpty y límites numéricos.

PASO 2.3.8 — Crear ValidationFilter genérico de Minimal API

Resultado de aprendizaje. Crear ValidationFilter genérico de Minimal API.

Procedimiento guiado.

1. Dentro de `Validacion` crea el archivo `ValidationFilter.cs` y copia la clase genérica completa del bloque C#: `ValidationFilter<T> : IEndpointFilter where T : class`.
2. Comprueba que el constructor recibe `IValidator<T>` y que `InvokeAsync` recibe `EndpointFilterInvocationContext` y `EndpointFilterDelegate`.
3. Conserva el flujo: localizar request, devolver error si falta, ejecutar `ValidateAsync`, agrupar errores si no es válido y finalmente llamar a `next(context)`.
4. Guarda y compila. **Todavía no se ejecutará en las peticiones HTTP**, porque falta registrarlo en los endpoints; no confundas existencia de la clase con conexión del filtro.

Archivo de trabajo: **`src/AceriaApp.Api/Validacion/ValidationFilter.cs`**.

```
using FluentValidation;

namespace AceriaApp.Api.Validacion;
```

```
public sealed class ValidationFilter<T>(IValidator<T> validator)
    : IEndpointFilter where T : class
{
    public async ValueTask<object?> InvokeAsync(
        EndpointFilterInvocationContext context, EndpointFilterDelegate next)
    {
        var request = context.Arguments.OfType<T>().FirstOrDefault();
        if (request is null)
            return Results.ValidationProblem(
                new Dictionary<string,string[]> {
                    ["body"] = new[] { "Falta el cuerpo." }
                });
        var result = await validator.ValidateAsync(
            request, context.HttpContext.RequestAborted);
        if (!result.IsValid)
        {
            var errors = result.Errors.GroupBy(x => x.PropertyName)
                .ToDictionary(x => x.Key,
                    x => x.Select(e => e.ErrorMessage).ToArray());
            return Results.ValidationProblem(errors);
        }
        return await next(context);
    }
}
```

Por qué funciona así. El filtro ejecuta ValidateAsync antes del manejador y utiliza Results.ValidationProblem si falla; solo invoca next(context) cuando la petición es válida.

Verificación concreta. El archivo del filtro genérico compila. Aún no debe afirmarse que POST, PUT y PATCH usen FluentValidation hasta completar 2.3.14–2.3.17.

Si algo falla. Si una petición inválida llega al endpoint, revisa el orden de ejecución y la condición IsValid antes de next(context).

PASO 2.3.9 — Obtener la petición de EndpointFilterInvocationContext

Resultado de aprendizaje. Obtener la petición de EndpointFilterInvocationContext.

Procedimiento guiado.

1. Abre `ValidationFilter.cs` y localiza dentro de `InvokeAsync` la instrucción `context.Arguments.OfType<T>().FirstOrDefault()`.
2. Analiza de dónde vendrán los argumentos del endpoint: GUID, contrato de entrada y dependencias inyectadas. Solo debe seleccionarse el argumento del tipo `T`.
3. Localiza la rama `if (request is null)` y comprueba que devuelve un error de validación asociado a `body`, no que invoque `next(context)`.
4. Prepara un punto de interrupción en esa línea para utilizarlo **después de conectar el filtro al POST** en 2.3.15. No inventes una ejecución del filtro antes de registrarlo.

Referencia de código: `src/AceriaApp.Api/Validacion/ValidationFilter.cs` y el endpoint de `Program.cs` sobre el que se instala el filtro. Comprueba dónde se produce `ValidationProblem` y dónde se llama a `next(context)`.

Por qué funciona así. El filtro ejecuta ValidateAsync antes del manejador y utiliza Results.ValidationProblem si falla; solo invoca next(context) cuando la petición es válida.

Verificación concreta. Puedes explicar qué request se extrae y qué respuesta se prepara si falta; el archivo sigue compilando sin añadir otro filtro.

Si algo falla. Si una petición inválida llega al endpoint, revisa el orden de ejecución y la condición IsValid antes de next(context).

PASO 2.3.10 — Ejecutar ValidateAsync y respetar RequestAborted

Resultado de aprendizaje. Ejecutar ValidateAsync y respetar RequestAborted.

Procedimiento guiado.

1. Sitúate en la línea `validator.ValidateAsync(request, context.HttpContext.RequestAborted)` del filtro.
2. Identifica el origen del `IValidator<T>` (DI), el objeto `request` que se validará y el token de cancelación de la petición HTTP.
3. No sustituyas `RequestAborted` por `CancellationToken.None`; conserva el resultado devuelto para comprobar `IsValid` en el siguiente bloque.
4. Deja preparada una petición inválida en `.http` y un punto de interrupción; la ejecución real se comprobará cuando el filtro quede asociado al endpoint.

Referencia de código: `src/AceriaApp.Api/Validacion/ValidationFilter.cs` y el endpoint de `Program.cs` sobre el que se instala el filtro. Comprueba dónde se produce `ValidationProblem` y dónde se llama a `next(context)`.

Por qué funciona así. El filtro ejecuta ValidateAsync antes del manejador y utiliza Results.ValidationProblem si falla; solo invoca next(context) cuando la petición es válida.

Verificación concreta. El método del filtro compila y mantiene la propagación del token `RequestAborted`; queda identificada la ruta de ejecución a verificar después.

Si algo falla. Si una petición inválida llega al endpoint, revisa el orden de ejecución y la condición IsValid antes de next(context).

PASO 2.3.11 — Agrupar errores por campo con LINQ

Resultado de aprendizaje. Agrupar errores por campo con LINQ.

Procedimiento guiado.

1. Abre la rama `if (!result.IsValid)` de `ValidationFilter.cs` y localiza `result.Errors.GroupBy(x => x.PropertyName)`.
2. Comprueba que los fallos se agrupan por el nombre de propiedad y que `ToDictionary` convierte cada grupo en un array `string[]` de mensajes.
3. Prepara un JSON que falle en dos campos, por ejemplo `cliente=""` y `cantidad=0`, para probarlo cuando el filtro se conecte.
4. No pegues otra copia del filtro: el fragmento de este paso muestra solo las instrucciones que ya escribiste en 2.3.8 y cuya estructura estás aprendiendo a interpretar.

Archivo de trabajo: **`src/AceriaApp.Api/Validacion/ValidationFilter.cs`**.

```
// Dentro de InvokeAsync, en la rama !result.IsValid:
var errors = result.Errors.GroupBy(x => x.PropertyName)
    .ToDictionary(x => x.Key,
        x => x.Select(e => e.ErrorMessage).ToArray());
return Results.ValidationProblem(errors);
```

Por qué funciona así. El filtro ejecuta ValidateAsync antes del manejador y utiliza Results.ValidationProblem si falla; solo invoca next(context) cuando la petición es válida.

Verificación concreta. El diccionario de validación podrá ofrecer una clave por campo con una lista de errores. En código sigue existiendo una sola clase `ValidationFilter<T>`.

Si algo falla. Si una petición inválida llega al endpoint, revisa el orden de ejecución y la condición IsValid antes de next(context).

PASO 2.3.12 — Responder mediante Results.ValidationProblem

Resultado de aprendizaje. Responder mediante Results.ValidationProblem.

Procedimiento guiado.

1. En la misma rama inválida identifica `return Results.ValidationProblem(errors)` y comprueba que está **antes** de `next(context)`.
2. Revisa con el depurador la diferencia entre devolver desde el filtro y permitir que el manejador guarde datos en LocalDB.
3. Tras conectar el filtro al POST (2.3.15), envía un cuerpo con Cliente vacío y comprueba estado 400 y propiedad `errors` en el JSON de respuesta.
4. Ejecuta un GET posterior a la colección y verifica que la petición rechazada no insertó una orden. No cambies este error por un 409 ni por un 422.

Referencia de código: `src/AceriaApp.Api/Validacion/ValidationFilter.cs` y el endpoint de `Program.cs` sobre el que se instala el filtro. Comprueba dónde se produce `ValidationProblem` y dónde se llama a `next(context)`.

Por qué funciona así. El filtro ejecuta ValidateAsync antes del manejador y utiliza Results.ValidationProblem si falla; solo invoca next(context) cuando la petición es válida.

Verificación concreta. La ejecución HTTP posterior a 2.3.15 debe mostrar 400, `errors` y ausencia de inserción, porque el filtro sale antes del manejador.

Si algo falla. Si una petición inválida llega al endpoint, revisa el orden de ejecución y la condición IsValid antes de next(context).

Caso práctico completo — Comprobar que el filtro realmente impide entrar al manejador

Arranca el proyecto 2.3 y abre `ValidationFilter.cs` y `Program.cs` al mismo tiempo. Coloca un breakpoint dentro de `InvokeAsync` y otro en la primera instrucción del manejador POST que crea la orden. Prepara una petición con Cliente vacío y los demás campos válidos. El deserializador de ASP.NET Core construirá CrearOrdenRequest porque el JSON tiene una estructura compatible; a continuación se ejecutará el filtro al que se asoció `AddEndpointFilter`.

Inspecciona el valor `request` en el depurador y observa cómo `ValidateAsync` devuelve `IsValid=false`. El filtro agrupa los `ValidationFailure` por nombre de propiedad y pasa el diccionario a `Results.ValidationProblem`. El manejador POST no debe ejecutarse, ni tampoco debe llamarse `SaveChangesAsync`. La respuesta HTTP contiene un 400 y un objeto JSON con la propiedad `errors`.

Repite con un cuerpo válido. El filtro volverá a ejecutarse, pero esta vez `IsValid` será verdadero y se llamará a `await next(context)`. El breakpoint del manejador POST debe alcanzarse. La orden se persiste y el servidor devuelve 201. Esta comparación entre las dos trazas es la demostración más importante del punto: crear una clase validadora y registrarla en DI no basta; hay que conectarla al endpoint que recibe la petición.

Comprueba después PUT y PATCH. El primero utiliza el contrato CrearOrdenRequest y el segundo ActualizarCantidadRequest. Cada uno debe utilizar el tipo genérico correcto al añadir el filtro. Un error común consiste en filtrar solamente POST y creer que todos los verbos quedan automáticamente protegidos. En ese caso PUT o PATCH aceptarían datos que POST rechaza.

Verificación: enviar Cantidad=0 debe devolver 400 con una entrada en errors; Cantidad=501 debe producir 422 porque es una regla del negocio.

PASO 2.3.13 — Permitir next(context) solo cuando IsValid

Resultado de aprendizaje. Permitir next(context) solo cuando IsValid.

Procedimiento guiado.

1. Localiza la última línea de `InvokeAsync`, `return await next(context);`, y observa que se alcanza **solo si `result.IsValid`**.
2. Revisa que en las ramas inválidas se devuelve inmediatamente un resultado y no hay un `next` adicional.
3. Después de asociar el filtro al POST, envía un request válido y comprueba con el depurador que entra en el manejador y llega a `SaveChangesAsync`.
4. Compara el resultado válido 201 con el inválido 400; ambos pasan por el filtro, pero solo el primero ejecuta la operación de negocio.

Referencia de código: `src/AceriaApp.Api/Validacion/ValidationFilter.cs` y el endpoint de `Program.cs` sobre el que se instala el filtro. Comprueba dónde se produce `ValidationProblem` y dónde se llama a `next(context)`.

Por qué funciona así. El filtro ejecuta ValidateAsync antes del manejador y utiliza Results.ValidationProblem si falla; solo invoca next(context) cuando la petición es válida.

Verificación concreta. El filtro no bloquea solicitudes válidas ni permite que las inválidas persistan. La comprobación práctica se realizará con POST filtrado.

Si algo falla. Si una petición inválida llega al endpoint, revisa el orden de ejecución y la condición IsValid antes de next(context).

PASO 2.3.14 — Registrar validadores en el contenedor DI

Resultado de aprendizaje. Registrar validadores en el contenedor DI.

Procedimiento guiado.

1. Abre `Program.cs` y añade los `using` de `AceriaApp.Api.Validacion` y `FluentValidation` al comienzo si aún no están presentes.
2. Antes de `var app = builder.Build();`, coloca `builder.Services.AddValidatorsFromAssemblyContaining<CrearOrdenValidator>();` exactamente como indica el fragmento.
3. Guarda y compila. Revisa que la aplicación puede resolver tanto `IValidator<CrearOrdenRequest>` como `IValidator<ActualizarCantidadRequest>`.
4. No supongas que el 400 ya sale del nuevo filtro: DI **solo registra** validadores; para ejecutarlos hacen falta las cadenas `AddEndpointFilter` de los tres pasos siguientes.

Archivo de trabajo: **`src/AceriaApp.Api/Program.cs`**.

```
using FluentValidation;
builder.Services.AddValidatorsFromAssemblyContaining<CrearOrdenValidator>();
```

Por qué funciona así. El registro de validadores en DI no ejecuta la validación. AddEndpointFilter debe asociarse a cada POST, PUT o PATCH que acepte ese contrato.

Verificación concreta. El registro está antes de `Build` y ambos validadores están en el ensamblado. Las rutas HTTP todavía conservan su semántica anterior.

Si algo falla. Si se acepta una petición inválida, puede faltar AddEndpointFilter aunque los validadores estén registrados en DI.

PASO 2.3.15 — Conectar filtro de CrearOrdenRequest al POST

Resultado de aprendizaje. Conectar filtro de CrearOrdenRequest al POST.

Procedimiento guiado.

1. Abre el `MapPost` de creación. Localiza la comprobación provisional de Cliente/Material/Cantidad introducida en 2.2; ahora se sustituye por FluentValidation.
2. Elimina esa comprobación provisional **tanto de POST como de PUT**, pero conserva el límite de negocio `if (request.Cantidad > 500)` que devuelve 422.
3. En la cadena del POST, después de `WithName` y `WithTags` y antes del `;`, añade `.AddEndpointFilter<ValidationFilter<CrearOrdenRequest>>()`.
4. Compila, envía un POST válido (debe responder 201) y otro con Cliente vacío (400 con `errors`). Observa que 501 sigue produciendo 422.

Archivo de trabajo: **`src/AceriaApp.Api/Program.cs`**.

```
.WithName("CrearOrden").WithTags("Ordenes")
.AddEndpointFilter<ValidationFilter<CrearOrdenRequest>>()

;
```

Por qué funciona así. El registro de validadores en DI no ejecuta la validación. AddEndpointFilter debe asociarse a cada POST, PUT o PATCH que acepte ese contrato.

Verificación concreta. POST filtrado devuelve 201 para válido, 400 `ValidationProblem` para inválido y 422 para capacidad; no hay dos validaciones contradictorias en el mismo endpoint.

Si algo falla. Si se acepta una petición inválida, puede faltar AddEndpointFilter aunque los validadores estén registrados en DI.

PASO 2.3.16 — Conectar filtro de CrearOrdenRequest al PUT

Resultado de aprendizaje. Conectar filtro de CrearOrdenRequest al PUT.

Procedimiento guiado.

1. Abre la cadena del **PUT existente** (`WithName("ReemplazarOrden")`), no crees otro `MapPut`.
2. Añade `.AddEndpointFilter<ValidationFilter<CrearOrdenRequest>>()` antes del punto y coma, conservando la ruta y el nombre.
3. Crea una orden pendiente y envía PUT con `Material=""`; revisa estado 400 y propiedad `errors`. Haz GET al mismo GUID para comprobar que la entidad no se modificó.
4. Envía después un PUT correcto y verifica 204 y modificación persistida mediante GET; esto comprueba que el filtro no impide el camino válido.

Referencia de código: `src/AceriaApp.Api/Validacion/ValidationFilter.cs` y el endpoint de `Program.cs` sobre el que se instala el filtro. Comprueba dónde se produce `ValidationProblem` y dónde se llama a `next(context)`.

Código que se incorpora en este paso:

```
.WithName("ReemplazarOrden").WithTags("Ordenes")
.AddEndpointFilter<ValidationFilter<CrearOrdenRequest>>();
```

Por qué funciona así. El registro de validadores en DI no ejecuta la validación. AddEndpointFilter debe asociarse a cada POST, PUT o PATCH que acepte ese contrato.

Verificación concreta. PUT tiene filtro activo, devuelve 400 para request inválido y 204 para correcto; el error no modifica la orden almacenada.

Si algo falla. Si se acepta una petición inválida, puede faltar AddEndpointFilter aunque los validadores estén registrados en DI.

PASO 2.3.17 — Conectar filtro de ActualizarCantidadRequest al PATCH

Resultado de aprendizaje. Conectar filtro de ActualizarCantidadRequest al PATCH.

Procedimiento guiado.

1. Abre el `MapPatch` especializado en cantidad. Elimina el `if (request.Cantidad <= 0)` provisional de la etapa anterior, porque ahora esa regla vive en `ActualizarCantidadValidator`.
2. En la cadena `.WithName("CambiarCantidad")`, añade `.AddEndpointFilter<ValidationFilter<ActualizarCantidadRequest>>()` antes del punto y coma.
3. Prueba PATCH con cantidad 0: debe responder 400 y no cambiar la orden. Repite con cantidad 501 y comprueba 422 de negocio.
4. Prueba con cantidad 20 en una orden pendiente y comprueba 200 con DTO y GET posterior, conservando Cliente y Material.

Archivo de trabajo: **`src/AceriaApp.Api/Program.cs`**.

```
.WithName("CambiarCantidad").WithTags("Ordenes")
.AddEndpointFilter<ValidationFilter<ActualizarCantidadRequest>>()

;
```

Por qué funciona así. El registro de validadores en DI no ejecuta la validación. AddEndpointFilter debe asociarse a cada POST, PUT o PATCH que acepte ese contrato.

Verificación concreta. POST, PUT y PATCH tienen sus filtros correctos y la política 422 sigue siendo independiente; los GET no llevan filtros de entrada de esos tipos.

Si algo falla. Si se acepta una petición inválida, puede faltar AddEndpointFilter aunque los validadores estén registrados en DI.

PASO 2.3.18 — Enviar POST incorrecto y comprobar status 400

Resultado de aprendizaje. Enviar POST incorrecto y comprobar status 400.

Procedimiento guiado.

1. Arranca la aplicación 2.3 y verifica que el archivo `.http` apunta al host correcto.
2. Envía un POST con `material` vacío y Cliente/Cantidad correctos. Inspecciona en la respuesta el código HTTP, `Content-Type` y el campo `errors`.
3. Anota que la petición llegó al filtro pero no a `SaveChangesAsync`: puedes comprobarlo con un punto de interrupción o consultando después el GET.
4. Repite con Material válido para comprobar que el camino normal conserva la creación 201 y `Location`.

Referencia de código: `src/AceriaApp.Api/Validacion/ValidationFilter.cs` y el endpoint de `Program.cs` sobre el que se instala el filtro. Comprueba dónde se produce `ValidationProblem` y dónde se llama a `next(context)`.

Por qué funciona así. Un 400 por datos fuera del rango, un 422 por la capacidad y un 409 por el estado de una orden pertenecen a causas diferentes que el cliente debe distinguir.

Verificación concreta. POST con Material vacío devuelve 400 con estructura de validación y no se inserta; POST correcto sigue creando órdenes.

Si algo falla. Si aparece 500 en vez de 400/422/409, revisa qué capa lanza la excepción y transforma el error de forma adecuada.

PASO 2.3.19 — Inspeccionar la propiedad errors del Problem Details

Resultado de aprendizaje. Inspeccionar la propiedad errors del Problem Details.

Procedimiento guiado.

1. Guarda el cuerpo JSON de un 400 producido por `ValidationFilter<T>`, no uno de JSON sintácticamente mal formado.
2. Localiza la propiedad `errors`, identifica la clave que corresponde al campo rechazado y cuenta sus mensajes.
3. Contrasta la representación de error con `Results.Problem(... 422)` enviado por la regla de capacidad: no son necesariamente idénticos.
4. Anota qué información es útil para un cliente y cuál no debe exponerse (detalles internos de excepciones o de conexión).

Referencia de código: `src/AceriaApp.Api/Validacion/ValidationFilter.cs` y el endpoint de `Program.cs` sobre el que se instala el filtro. Comprueba dónde se produce `ValidationProblem` y dónde se llama a `next(context)`.

Por qué funciona así. Un 400 por datos fuera del rango, un 422 por la capacidad y un 409 por el estado de una orden pertenecen a causas diferentes que el cliente debe distinguir.

Verificación concreta. Dispones de un ejemplo concreto de error 400 por propiedad y sabes distinguirlo de un conflicto 409 y de un 422 de negocio.

Si algo falla. Si aparece 500 en vez de 400/422/409, revisa qué capa lanza la excepción y transforma el error de forma adecuada.

PASO 2.3.20 — Probar límite 10001 frente a la capacidad 501

Resultado de aprendizaje. Probar límite 10001 frente a la capacidad 501.

Procedimiento guiado.

1. Envía un POST con `cantidad: 10001` y datos de Cliente/Material válidos; guarda código y respuesta.
2. Envía otra petición con `cantidad: 501`, manteniendo idénticos los demás campos para aislar la causa.
3. Comprueba en el depurador que 10001 falla en FluentValidation antes del manejador, mientras 501 pasa el validador pero activa `Cantidad > 500`.
4. Ejecuta GET de colección para asegurar que ninguna de las dos solicitudes rechazadas insertó órdenes.

Referencia de código: `src/AceriaApp.Api/Validacion/ValidationFilter.cs` y el endpoint de `Program.cs` sobre el que se instala el filtro. Comprueba dónde se produce `ValidationProblem` y dónde se llama a `next(context)`.

Por qué funciona así. Un 400 por datos fuera del rango, un 422 por la capacidad y un 409 por el estado de una orden pertenecen a causas diferentes que el cliente debe distinguir.

Verificación concreta. 10001 → 400 con `errors`, 501 → 422 con Problem Details. Ambas entradas dejan inalterada la tabla de órdenes.

Si algo falla. Si aparece 500 en vez de 400/422/409, revisa qué capa lanza la excepción y transforma el error de forma adecuada.

PASO 2.3.21 — Reproducir 409 con orden ya finalizada

Resultado de aprendizaje. Reproducir 409 con orden ya finalizada.

Procedimiento guiado.

1. Crea una orden con datos válidos y finalízala mediante `POST /api/v1/ordenes/{id}/finalizar`.
2. Intenta después modificarla con PUT y un cuerpo JSON válido; no uses datos inválidos que pudieran devolver 400 antes de evaluar el estado.
3. Comprueba **409 Conflict** y ejecuta GET del recurso para verificar que sigue finalizado y no se han modificado sus campos.
4. Contrasta el significado de 409 (conflicto de estado persistido) con 400 (datos inválidos) y 422 (capacidad excedida).

Por qué funciona así. Un 400 por datos fuera del rango, un 422 por la capacidad y un 409 por el estado de una orden pertenecen a causas diferentes que el cliente debe distinguir.

Verificación concreta. PUT sobre una orden finalizada devuelve 409 y no altera el registro. Las reglas de estado del punto 2.2 siguen activas.

Si algo falla. Si aparece 500 en vez de 400/422/409, revisa qué capa lanza la excepción y transforma el error de forma adecuada.

PASO 2.3.22 — Crear pruebas de validación mediante HTTP

Resultado de aprendizaje. Crear pruebas de validación mediante HTTP.

Procedimiento guiado.

1. En `tests/AceriaApp.Api.Tests` crea `ValidacionHttpTests.cs` y utiliza el **archivo C# completo** que aparece en el laboratorio, con sus imports y una sola clase pública.
2. Identifica la prueba parametrizada con tres `[InlineData]` y los dos hechos restantes: cantidad 10001 → 400 y cantidad 501 → 422.
3. Comprueba que las aserciones leen HTTP real por `ApiFactory` y que, para errores 400, el JSON contiene `errors`; no se trata de comprobar solo que existe una clase Validator.
4. Ejecuta `dotnet test tests/AceriaApp.Api.Tests/AceriaApp.Api.Tests.csproj` y verifica que todos los tests previos siguen funcionando junto con los nuevos casos.

Archivo de trabajo: `tests/AceriaApp.Api.Tests/ValidacionHttpTests.cs`.

```
using System.Net;
using System.Net.Http.Json;
using System.Text.Json;
using AceriaApp.Api.Contratos;
using Xunit;
namespace AceriaApp.Api.Tests;
// INICIO PASO 2.3.22: se verifica el filtro ACTIVO, no su mera existencia.
public class ValidacionHttpTests
{
    [Theory]
    [InlineData("","CHAPA",10)]
    [InlineData("ACX","",10)]
    [InlineData("ACX","CHAPA",0)]
    public async Task DatosInvalidos_Es400ConErrors(string cliente,string material,int cantidad)
    {
        using var f=new ApiFactory();
        var r=await f.CreateClient().PostAsJsonAsync("/api/v1/ordenes",
            new CrearOrdenRequest(cliente,material,cantidad));
        Assert.Equal(HttpStatusCode.BadRequest,r.StatusCode);
        using var json=JsonDocument.Parse(await r.Content.ReadAsStringAsync());
        Assert.True(json.RootElement.TryGetProperty("errors",out _));
    }
    [Fact] public async Task CantidadMasDe10000_Es400()
    {
        using var f=new ApiFactory();
        var r=await f.CreateClient().PostAsJsonAsync("/api/v1/ordenes",
            new CrearOrdenRequest("ACX","CHAPA",10001));
```

```
        Assert.Equal(HttpStatusCode.BadRequest,r.StatusCode);
    }
    [Fact] public async Task ReglaDeNegocio_501_Es422()
    {
        using var f=new ApiFactory();
        var r=await f.CreateClient().PostAsJsonAsync("/api/v1/ordenes",
            new CrearOrdenRequest("ACX","CHAPA",501));
        Assert.Equal(HttpStatusCode.UnprocessableEntity,r.StatusCode);
    }
}
```

Por qué funciona así. Las pruebas HTTP recorren el pipeline completo y comprueban que los fallos se comunican como Problem Details sin alterar los datos.

Verificación concreta. Las pruebas de 2.3 deben elevar el total acumulado a **12 superadas**, sin eliminar ni modificar las de GET y escritura.

Si algo falla. Si el test solo instancia un record, no prueba HTTP; usa WebApplicationFactory y examina errors del JSON.

PASO 2.3.23 — Verificar que los GET siguen sin regresiones

Resultado de aprendizaje. Verificar que los GET siguen sin regresiones.

Procedimiento guiado.

1. En Visual Studio abre **Prueba > Explorador de pruebas** o ejecuta `dotnet test` desde la carpeta raíz de 2.3.
2. Agrupa los resultados en tres familias: `GetTests`, `EscrituraTests` y `ValidacionHttpTests`.
3. Comprueba que el total es 12 y que no aparecen pruebas desconocidas creadas por plantillas, ni pruebas de exploración temporales de 2.3.7.
4. Ejecuta por separado una prueba GET y una de escritura si hay algún fallo para distinguir regresión del código previo de problema de validación.

Por qué funciona así. Las pruebas HTTP recorren el pipeline completo y comprueban que los fallos se comunican como Problem Details sin alterar los datos.

Verificación concreta. `dotnet test` termina con 12 superadas y 0 fallidas; la introducción de filtros no ha roto las rutas de lectura ni las operaciones de escritura.

Si algo falla. Si el test solo instancia un record, no prueba HTTP; usa WebApplicationFactory y examina errors del JSON.

PASO 2.3.24 — Reto: personalizar mensajes y comprobar errores

Resultado de aprendizaje. Reto: personalizar mensajes y comprobar errores.

Procedimiento guiado.

1. Reto opcional: realiza los cambios en una copia de tu solución de 2.3 para no alterar el estado de referencia.
2. En una regla FluentValidation concreta añade `.WithMessage("La cantidad debe ser mayor que cero")` o un mensaje equivalente ajustado a la propiedad.
3. Envía una petición inválida y localiza el texto personalizado dentro de `errors` del JSON 400; compara antes y después.
4. Añade una aserción HTTP sobre esa cadena, ejecútala y documenta el resultado. Revierte el reto en la copia que utilizarás para comenzar 2.4.

Por qué funciona así. Las pruebas HTTP recorren el pipeline completo y comprueban que los fallos se comunican como Problem Details sin alterar los datos.

Verificación concreta. El reto muestra el mensaje personalizado en la respuesta del servicio sin cambiar su estado HTTP 400; el proyecto base conserva sus tests habituales.

Si algo falla. Si el test solo instancia un record, no prueba HTTP; usa WebApplicationFactory y examina errors del JSON.

Caso práctico completo — Del error 400 al 409: tres pruebas con causas diferentes

Para comprobar la separación de responsabilidades, prepara primero un POST de cantidad 10.001. Es un número entero válido desde el punto de vista sintáctico, pero infringe la regla `InclusiveBetween(1,10000)`. Por tanto, FluentValidation genera un error semántico y el filtro devuelve 400. Inspecciona el campo `errors`: debe identificar Cantidad. No necesitas consultar SQL Server para determinar que un valor está fuera de ese intervalo.

Envía después exactamente el mismo cuerpo, cambiando Cantidad a 501. El validador semántico deja pasar el valor, porque 501 pertenece al rango de entrada. El manejador POST comprueba entonces la capacidad del taller y decide responder 422. La decisión podría depender en el futuro de inventario, estado de las máquinas o política comercial; por esa razón no es idéntica a una comprobación de formato. Este es un contrato elegido para el ejercicio, no una regla universal que obligue a todas las APIs a usar 422.

La tercera prueba necesita un recurso persistido. Crea una orden con cantidad 10, recupera su Guid, ejecútale `/finalizar` y después envía un PUT de modificación. El servidor devuelve 409 porque el recurso está en un estado incompatible con la modificación. Un 409 no significa que el tipo `int` de Cantidad sea inválido: significa que el recurso existente no admite la operación solicitada.

Escribe pruebas en `ValidacionHttpTests.cs` que cubran las tres situaciones. Usa `WebApplicationFactory` para hacer peticiones HTTP reales y comprobar el código que recibe un consumidor. La prueba de 400 debe revisar `errors`; las de 422 y 409 comprueban la distinción de política y estado.

Control de regresión: después de añadir filtros, vuelve a ejecutar GET de lista y detalle. Ninguna regla de validación debería convertir una lectura válida en un fallo de contrato.

Matriz de comprobación del punto

Comportamiento	Resultado esperado
`Cliente vacío`	400 + errors
`Cantidad 10001`	400 + errors
`Cantidad 501`	422
`Orden finalizada`	409

Reto integrador. Reconstruye sin consultar la solución el último endpoint o validación practicada y añade una prueba HTTP que demuestre su resultado. Describe el recurso, el verbo, el contrato y el código devuelto. Si no puedes reproducir una respuesta, vuelve al primer paso que introdujo el comportamiento.

Punto 2.4 — Documento OpenAPI generado y comprobado con Swagger

Proyecto comprobado: `PROYECTO/2.4-documentacion-openapi`. El punto comienza estudiando el contrato de recursos y concluye con una ejecución verificable. Ninguna comprobación sustituye a un endpoint realmente implementado.

PASO 2.4.1 — Comprobar el contrato REST que se va a documentar

Resultado de aprendizaje. Comprobar el contrato REST que se va a documentar.

Procedimiento guiado.

1. Crea una carpeta de trabajo 2.4 a partir de tu **solución 2.3 completada** y ejecuta sus 12 tests antes de añadir OpenAPI.
2. Abre `Program.cs` y enumera todas las operaciones: dos GET, POST de creación, PUT, PATCH de cantidad, DELETE y POST finalizar.
3. Para cada handler identifica el DTO de salida o contrato de entrada y sus códigos de éxito/error tal como aparecen en las ramas de ejecución.
4. Prepara una tabla que servirá para comparar **lo que hace el código** con **lo que describirá el documento Swagger**. No inventes rutas que no existan.

Por qué funciona así. Swashbuckle descubre la API real desde ASP.NET Core: registrar AddSwaggerGen y definir SwaggerDoc produce un documento estándar a partir de rutas y tipos.

Verificación concreta. La tabla contiene siete operaciones y sus códigos reales, y el proyecto de 2.3 sigue pasando 12 tests antes de introducir Swagger.

Si algo falla. Si SwaggerGen no se resuelve, revisa la versión de Swashbuckle compatible con .NET 8.

PASO 2.4.2 — Instalar Swashbuckle.AspNetCore compatible con .NET 8

Resultado de aprendizaje. Instalar Swashbuckle.AspNetCore compatible con .NET 8.

Procedimiento guiado.

1. En Visual Studio, botón derecho sobre el proyecto API > **Administrar paquetes NuGet**; busca `Swashbuckle.AspNetCore`.
2. Instala la versión `6.6.2`, utilizada por la referencia .NET 8. Como alternativa ejecuta `dotnet add src/AceriaApp.Api/AceriaApp.Api.csproj package Swashbuckle.AspNetCore --version 6.6.2`.
3. Abre `AceriaApp.Api.csproj` y verifica que conserva EF Core SQL Server y FluentValidation además de Swashbuckle.
4. Restaura con `dotnet restore`. No instales Swashbuckle exclusivamente en el proyecto de tests ni añadas servicios Swagger antes de comprender cómo se registran.

Por qué funciona así. Swashbuckle descubre la API real desde ASP.NET Core: registrar AddSwaggerGen y definir SwaggerDoc produce un documento estándar a partir de rutas y tipos.

Verificación concreta. El proyecto API tiene las tres dependencias necesarias y restaura NuGet. Todavía no debería responder `/swagger/v1/swagger.json` hasta configurar los servicios y el middleware.

Si algo falla. Si SwaggerGen no se resuelve, revisa la versión de Swashbuckle compatible con .NET 8.

PASO 2.4.3 — Crear carpeta Documentacion y extensiones de configuración

Resultado de aprendizaje. Crear carpeta Documentacion y extensiones de configuración.

Procedimiento guiado.

1. En `src/AceriaApp.Api` crea una carpeta `Documentacion` mediante **Agregar > Nueva carpeta**.
2. Crea en ella `DocumentacionExtensions.cs`. Reemplaza el archivo inicial por el namespace `AceriaApp.Api.Documentacion` y la clase `public static class DocumentacionExtensions` del ejemplo.

3. Deja por ahora únicamente la estructura de la clase: aún no insertes el generador, el middleware ni el filtro de ejemplo.
4. Guarda y compila; confirma que el namespace se corresponde con el directorio donde vive el archivo, y no se declara por accidente otra clase del mismo nombre.

Archivo de trabajo: `src/AceriaApp.Api/Documentacion/DocumentacionExtensions.cs`.

```
// Crea src/AceriaApp.Api/Documentacion/DocumentacionExtensions.cs:
namespace AceriaApp.Api.Documentacion;

public static class DocumentacionExtensions
{
    // En 2.4.4 se añadirá AddDocumentacion.
    // En 2.4.7 se añadirá UseDocumentacion.
}
```

Por qué funciona así. Swashbuckle descubre la API real desde ASP.NET Core: registrar AddSwaggerGen y definir SwaggerDoc produce un documento estándar a partir de rutas y tipos.

Verificación concreta. `DocumentacionExtensions.cs` compila como clase estática vacía y no altera los endpoints existentes de órdenes.

Si algo falla. Si SwaggerGen no se resuelve, revisa la versión de Swashbuckle compatible con .NET 8.

PASO 2.4.4 — Registrar AddEndpointsApiExplorer y AddSwaggerGen

Resultado de aprendizaje. Registrar AddEndpointsApiExplorer y AddSwaggerGen.

Procedimiento guiado.

1. Abre `DocumentacionExtensions.cs` y sitúa el cursor **dentro** de la clase, antes de la última llave.
2. Inserta el método estático de extensión `AddDocumentacion(this IServiceCollection services)` del bloque del paso; debe devolver el mismo objeto `services`.
3. En su cuerpo añade `services.AddEndpointsApiExplorer()` y `services.AddSwaggerGen()`, en ese orden, sin eliminar el método de extensión.
4. Guarda y compila. Observa que todavía no se llama a `AddDocumentacion` desde `Program.cs` ni se ha definido un middleware visible en navegador.

Archivo de trabajo: `src/AceriaApp.Api/Documentacion/DocumentacionExtensions.cs`.

```
// Dentro de DocumentacionExtensions:
public static IServiceCollection AddDocumentacion(
    this IServiceCollection services)
{
    services.AddEndpointsApiExplorer();
    services.AddSwaggerGen();
    return services;
}
```

Por qué funciona así. Swashbuckle descubre la API real desde ASP.NET Core: registrar AddSwaggerGen y definir SwaggerDoc produce un documento estándar a partir de rutas y tipos.

Verificación concreta. El proyecto compila con `AddDocumentacion` disponible; Swagger aún no está publicado porque faltan la conexión y el middleware.

Si algo falla. Si SwaggerGen no se resuelve, revisa la versión de Swashbuckle compatible con .NET 8.

PASO 2.4.5 — Configurar SwaggerDoc con información y versión v1

Resultado de aprendizaje. Configurar SwaggerDoc con información y versión v1.

Procedimiento guiado.

1. En el mismo `AddDocumentacion`, encuentra la línea `services.AddSwaggerGen();` y sustitúyela por el bloque `AddSwaggerGen(options => { ... })` del manual.
2. Añade `using Microsoft.OpenApi.Models;` arriba del archivo para resolver `OpenApiInfo` y `OpenApiContact`.
3. Comprueba `SwaggerDoc("v1", ...)`: título `AceriaApp — Órdenes de producción`, versión `v1`, descripción y contacto. No cambies el nombre a `v2` ni lo confundas con versionado automático de rutas.
4. Guarda y compila. No registres aún `SchemaFilter<EjemploCrearOrden>` porque esa clase se creará en el paso 2.4.16.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Código de este paso. En el mismo archivo agrega `using Microsoft.OpenApi.Models;`. El documento se llama `v1`: eso no crea automáticamente otra versión de la API.

```
// Sustituye services.AddSwaggerGen() dentro de AddDocumentacion:
services.AddSwaggerGen(options =>
{
    options.SwaggerDoc("v1", new OpenApiInfo
    {
        Title = "AceriaApp — Órdenes de producción",
        Version = "v1",
        Description = "API REST educativa creada con ASP.NET Core 8.",
        Contact = new OpenApiContact
        {
            Name = "Equipo AceriaApp",
            Email = "equipo@acerialocal"
        }
    });
});
```

Por qué funciona así. Swashbuckle descubre la API real desde ASP.NET Core: registrar AddSwaggerGen y definir SwaggerDoc produce un documento estándar a partir de rutas y tipos.

Verificación concreta. `AddDocumentacion` configura un único documento OpenAPI `v1` y compila; todavía no crea versiones adicionales de los endpoints.

Si algo falla. Si SwaggerGen no se resuelve, revisa la versión de Swashbuckle compatible con .NET 8.

PASO 2.4.6 — Conectar AddDocumentacion en Program.cs

Resultado de aprendizaje. Conectar AddDocumentacion en Program.cs.

Procedimiento guiado.

1. Abre `Program.cs` y agrega `using AceriaApp.Api.Documentacion;` al principio.
2. Busca la zona de registro de servicios, **antes de `var app = builder.Build();`**, junto a `AddDbContext` y `AddValidatorsFromAssemblyContaining`.
3. Escribe una sola línea `builder.Services.AddDocumentacion();` como indica el fragmento del laboratorio.
4. Guarda y compila. Verifica que los servicios Swagger están registrados y los endpoints de negocio siguen existiendo; todavía falta llamar al método que publica Swagger después de `Build`.

Archivo de trabajo: **`src/AceriaApp.Api/Program.cs`**.

```
builder.Services.AddDocumentacion();
```

Por qué funciona así. La publicación de Swagger UI y JSON ocurre después de construir la aplicación y se restringe a Development; la documentación no debe inventarse manualmente.

Verificación concreta. Una llamada a `AddDocumentacion` está antes de `Build`; no debe aparecer otro `builder.Build()` ni duplicación del registro.

Si algo falla. Si /swagger devuelve 404 en Development, comprueba AddDocumentacion y UseDocumentacion y el entorno.

PASO 2.4.7 — Configurar UseSwagger y Swagger UI en Development

Resultado de aprendizaje. Configurar UseSwagger y Swagger UI en Development.

Procedimiento guiado.

1. En `DocumentacionExtensions.cs`, dentro de la clase estática y después de `AddDocumentacion`, añade el segundo método de extensión `UseDocumentacion(this WebApplication app)`.
2. Copia el bloque del laboratorio con `if (app.Environment.IsDevelopment())`, `app.UseSwagger()` y `app.UseSwaggerUI(...)`.
3. Verifica que la UI apunta a `"/swagger/v1/swagger.json"` y que el método devuelve `app` para permitir el encadenamiento.
4. Compila. Explica qué cambia al configurar el entorno `Production`: no se ejecutarán estos middleware aunque el paquete Swashbuckle esté instalado.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Código de este paso. Incorpora el método en la misma clase estática. La restricción a `Development` debe ser comprobada por un test posterior.

```
// Añade a la misma clase DocumentacionExtensions:
public static WebApplication UseDocumentacion(this WebApplication app)
{
    if (app.Environment.IsDevelopment())
    {
        app.UseSwagger();
        app.UseSwaggerUI(o =>
            o.SwaggerEndpoint("/swagger/v1/swagger.json", "AceriaApp v1"));
    }
    return app;
}
```

Por qué funciona así. La publicación de Swagger UI y JSON ocurre después de construir la aplicación y se restringe a Development; la documentación no debe inventarse manualmente.

Verificación concreta. `UseDocumentacion` compila y solo publica Swagger/Swagger UI cuando `IsDevelopment()` es verdadero.

Si algo falla. Si /swagger devuelve 404 en Development, comprueba AddDocumentacion y UseDocumentacion y el entorno.

PASO 2.4.8 — Conectar UseDocumentacion y abrir /swagger

Resultado de aprendizaje. Conectar UseDocumentacion y abrir /swagger.

Procedimiento guiado.

1. Abre `Program.cs`, busca `app.Run();` y sitúa el cursor **inmediatamente antes de esa línea**.
2. Inserta `app.UseDocumentacion();` una sola vez después del registro de los endpoints. No la confundas con `builder.Services.AddDocumentacion()` de 2.4.6.

3. Comprueba que el perfil `ASPNETCORE_ENVIRONMENT` es Development y arranca la API desde Visual Studio o con `dotnet run`.
4. Abre `http://localhost:PUERTO_REAL/swagger` usando la URL que emite Kestrel; verifica que Swagger UI muestra los endpoints de órdenes.

Archivo de trabajo: **`src/AceriaApp.Api/Program.cs`**.

```
app.UseDocumentacion(); // PASO 2.4.4
```

Por qué funciona así. La publicación de Swagger UI y JSON ocurre después de construir la aplicación y se restringe a Development; la documentación no debe inventarse manualmente.

Verificación concreta. Swagger UI abre en Development y enumera las rutas existentes; si se obtiene 404, comprueba el entorno, la URL exacta y la invocación `UseDocumentacion`.

Si algo falla. Si /swagger devuelve 404 en Development, comprueba AddDocumentacion y UseDocumentacion y el entorno.

PASO 2.4.9 — Descargar el JSON real /swagger/v1/swagger.json

Resultado de aprendizaje. Descargar el JSON real /swagger/v1/swagger.json.

Procedimiento guiado.

1. Mantén la API ejecutándose en Development y copia la URL del servidor a tu variable `@Host` en `AceriaApp.http`.
2. Añade y ejecuta la solicitud `GET {{Host}}/swagger/v1/swagger.json` mediante **Send Request**; no escribas un documento YAML manual para sustituirla.
3. Confirma que el servidor devuelve HTTP 200, tipo JSON y un objeto que contiene `openapi`, `info`, `paths` y `components` cuando corresponda.
4. Guarda la respuesta generada para compararla con cambios posteriores en `WithSummary`, schemas y ejemplos.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Por qué funciona así. La publicación de Swagger UI y JSON ocurre después de construir la aplicación y se restringe a Development; la documentación no debe inventarse manualmente.

Verificación concreta. La evidencia es el **JSON real** devuelto por ASP.NET Core/Swashbuckle con 200. El contenido procede del código ejecutable, no de un archivo estático.

Si algo falla. Si /swagger devuelve 404 en Development, comprueba AddDocumentacion y UseDocumentacion y el entorno.

PASO 2.4.10 — Identificar info, paths, components y schemas

Resultado de aprendizaje. Identificar info, paths, components y schemas.

Procedimiento guiado.

1. Abre en un editor con formato JSON la respuesta obtenida en 2.4.9 (Visual Studio permite plegar los nodos).
2. Localiza `info.version` y comprueba que vale `v1`. Busca después `paths` y enumera las rutas de órdenes, teniendo en cuenta que una ruta puede contener varios verbos.
3. Expande `components.schemas` y busca los contratos de entrada/salida generados a partir de los tipos C# de la API.
4. En tu tabla de comparación de 2.4.1 marca qué elementos fueron descubiertos automáticamente y cuáles necesitarán metadatos o ejemplos añadidos manualmente.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Por qué funciona así. La publicación de Swagger UI y JSON ocurre después de construir la aplicación y se restringe a Development; la documentación no debe inventarse manualmente.

Verificación concreta. Puedes señalar en el documento generado la versión `v1`, las rutas de negocio y los esquemas; no debes confundir `info.version` con el segmento literal `/api/v1`.

Si algo falla. Si /swagger devuelve 404 en Development, comprueba AddDocumentacion y UseDocumentacion y el entorno.

PASO 2.4.11 — Aplicar WithName y WithTags a los endpoints

Resultado de aprendizaje. Aplicar WithName y WithTags a los endpoints.

Procedimiento guiado.

1. En `Program.cs`, localiza para cada endpoint la cadena que empieza por `.WithName(...)` y continúa con `.WithTags("Ordenes")`.
2. Comprueba que los nombres como `ListarOrdenes`, `ObtenerOrdenPorId`, `CrearOrden` y `CambiarCantidad` son distintos entre sí.
3. Vuelve a Swagger UI y comprueba que las operaciones aparecen agrupadas bajo `Ordenes` debido a la etiqueta indicada.
4. No añadas un segundo `WithName` ni registres nuevas rutas: estos metadatos ya existen en el código heredado y se estudian aquí antes de ampliarlos.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Por qué funciona así. Los metadatos de una operación declaran rutas, respuestas y esquemas; son documentación de un comportamiento que ya fue implementado y probado.

Verificación concreta. Todas las operaciones existentes conservan un nombre y la etiqueta `Ordenes`; no se duplican endpoints ni nombres.

Si algo falla. Si falta un código de respuesta, añade Produces/ProducesProblem al endpoint correspondiente sin inventar comportamientos.

PASO 2.4.12 — Documentar GET con WithSummary y Produces

Resultado de aprendizaje. Documentar GET con WithSummary y Produces.

Procedimiento guiado.

1. Abre el GET de colección y localiza su cadena `.WithName("ListarOrdenes")...`. Añade `WithSummary` y `Produces<IEnumerable<OrdenDto>>(200)` tal como se muestra.
2. En el GET individual añade el segundo fragmento del mismo paso: `WithSummary("Obtiene una orden por ID")`, `Produces<OrdenDto>(200)` y `.Produces(404)`.
3. No modifiques los delegados que consultan EF Core: ahora trabajas únicamente sobre **metadatos de contrato**.
4. Compila, descarga otra vez el JSON OpenAPI y revisa que ambos GET anuncian los códigos y tipos esperados.

Archivo de trabajo: **`src/AceriaApp.Api/Program.cs`**.

```
.WithName("ListarOrdenes").WithTags("Ordenes")
.WithSummary("Lista las órdenes de producción").Produces<IEnumerable<OrdenDto>>(200)
;
```

Metadatos del GET individual (el mismo paso):

```
.WithName("ObtenerOrdenPorId").WithTags("Ordenes")
.WithSummary("Obtiene una orden por ID")
.Produces<OrdenDto>(200).Produces(404);
```

Por qué funciona así. Los metadatos de una operación declaran rutas, respuestas y esquemas; son documentación de un comportamiento que ya fue implementado y probado.

Verificación concreta. Los dos GET mantienen su comportamiento 200/404 y su documentación describe `OrdenDto`, no la entidad `Orden` del contexto.

Si algo falla. Si falta un código de respuesta, añade Produces/ProducesProblem al endpoint correspondiente sin inventar comportamientos.

Caso práctico completo — Desde MapGet hasta paths: interpretar Swagger JSON

Inicia la API 2.4 en Development y abre la URL `http://localhost:5082/swagger/v1/swagger.json`. Este documento no se guarda a mano en la carpeta docs. Lo produce SwaggerGen cuando la aplicación solicita información sobre los endpoints de ASP.NET Core. Busca la propiedad `info`: debe contener el título de AceriaApp y la versión v1 que registró `SwaggerDoc`.

Continúa con la sección `paths`. Busca `/api/v1/ordenes` y despliega sus operaciones `get` y `post`. Cada operación representa un manejador declarado mediante MapGet o MapPost. La sección muestra los tipos de cuerpo de entrada y las respuestas descritas con `Produces`, `ProducesProblem` y otros metadatos. Una ruta ausente en el documento puede indicar que no está registrada, que la exploración no está configurada o que sus metadatos son incompletos. No lo arregles escribiendo una ruta ficticia en un YAML separado.

Abre a continuación Swagger UI en `/swagger`. La interfaz interpreta el JSON y permite visualizar los endpoints agrupados con `WithTags`. Selecciona GET de colección, ejecuta Try it out y compara el estado 200 con el del archivo HTTP del alumno. Las dos herramientas consumen la misma API; Swagger UI no proporciona un servidor de ejemplo independiente.

En `Program.cs` revisa `WithSummary` y `Produces`. Añade o corrige el resumen del GET y vuelve a descargar el JSON: el metadato cambia porque la fuente de la documentación es el código. Esta relación bidireccional entre implementar y documentar es precisamente el aprendizaje del punto 2.4.

Error habitual: configurar AddSwaggerGen pero olvidar `app.UseDocumentacion()` después de Build. En ese caso los servicios pueden estar registrados y, sin embargo, /swagger devolver 404.

PASO 2.4.13 — Documentar POST con 201 y ValidationProblem 400

Resultado de aprendizaje. Documentar POST con 201 y ValidationProblem 400.

Procedimiento guiado.

1. Abre el POST `CrearOrden` y coloca el cursor en su cadena fluida `.WithName("CrearOrden")...`, no dentro de la lambda.
2. Conserva el filtro `.AddEndpointFilter<ValidationFilter<CrearOrdenRequest>>()` introducido en 2.3.
3. Añade el resumen y las respuestas del fragmento: `Produces<OrdenDto>(201)`, `ProducesValidationProblem(400)` y `ProducesProblem(422)`.
4. Guarda y compila. Ejecuta un POST válido, otro inválido y otro de capacidad 501; compara las respuestas efectivas con los códigos anunciados en el JSON Swagger.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Código que se incorpora en este paso:

```
.WithName("CrearOrden").WithTags("Ordenes")
.AddEndpointFilter<ValidationFilter<CrearOrdenRequest>>()
.WithSummary("Crea una orden")
.Produces<OrdenDto>(201)
.ProducesValidationProblem(400)
.ProducesProblem(422);
```

Por qué funciona así. Los metadatos de una operación declaran rutas, respuestas y esquemas; son documentación de un comportamiento que ya fue implementado y probado.

Verificación concreta. POST anuncia 201, 400 y 422 y sigue ejecutando exactamente las mismas ramas de creación, validación y capacidad que en 2.3.

Si algo falla. Si falta un código de respuesta, añade Produces/ProducesProblem al endpoint correspondiente sin inventar comportamientos.

PASO 2.4.14 — Documentar PUT y PATCH con códigos alternativos

Resultado de aprendizaje. Documentar PUT y PATCH con códigos alternativos.

Procedimiento guiado.

1. En `Program.cs`, localiza el PUT llamado `ReemplazarOrden` y añade la cadena documental **completa para PUT** del bloque de este paso, conservando su filtro.
2. Comprueba que declara 204 (éxito), 404 (ausente), 400 (validación), 409 (estado) y 422 (capacidad). No confundas `ProducesProblem(422)` con ejecutar la validación.
3. Localiza el PATCH `CambiarCantidad`; añade la **segunda cadena**, específica de PATCH, con 200 y los códigos alternativos correspondientes.
4. Compila y contrasta en `/swagger/v1/swagger.json` las respuestas de PUT y PATCH. Recuerda que `Produces` documenta: no cambia lo que devuelve el delegado.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Código que se incorpora en este paso:

```
.WithName("ReemplazarOrden").WithTags("Ordenes")
.AddEndpointFilter<ValidationFilter<CrearOrdenRequest>>()
.WithSummary("Sustituye campos editables")
.Produces(204).Produces(404)
.ProducesProblem(409)
.ProducesValidationProblem(400)
.ProducesProblem(422);

// La cadena de PATCH se ofrece separadamente a continuación.
```

Declaración completa de PATCH, separada de PUT:

```
.WithName("CambiarCantidad").WithTags("Ordenes")
.AddEndpointFilter<ValidationFilter<ActualizarCantidadRequest>>()
.WithSummary("Modifica cantidad").Produces<OrdenDto>(200).Produces(404)
.ProducesValidationProblem(400).ProducesProblem(409).ProducesProblem(422);
```

Por qué funciona así. Los metadatos de una operación declaran rutas, respuestas y esquemas; son documentación de un comportamiento que ya fue implementado y probado.

Verificación concreta. PUT anuncia 204/404/400/409/422 y PATCH anuncia 200/404/400/409/422; sus handlers conservan la misma persistencia.

Si algo falla. Si falta un código de respuesta, añade Produces/ProducesProblem al endpoint correspondiente sin inventar comportamientos.

PASO 2.4.15 — Documentar DELETE y el conflicto 409

Resultado de aprendizaje. Documentar DELETE y el conflicto 409.

Procedimiento guiado.

1. En el `MapDelete` llamado `EliminarOrden`, añade los metadatos del primer bloque: 204 para borrado, 404 si no existe y 409 si está finalizada.
2. En el POST de transición `FinalizarOrden`, incorpora la segunda cadena del paso con 200, 404 y 409.
3. Vuelve a generar el documento Swagger y busca ambas operaciones dentro de `paths`; verifica que cada código declarado corresponde a un camino que realmente puede ocurrir.
4. Ejecuta un DELETE de orden finalizada y una repetición de finalizar para observar el 409 antes de dar por documentado el contrato.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Código que se incorpora en este paso:

```
.WithName("EliminarOrden").WithTags("Ordenes")
.WithSummary("Elimina una orden pendiente")
.Produces(204).Produces(404)
.ProducesProblem(409);

// La cadena de finalizar se ofrece separadamente a continuación.
```

Declaración de la transición finalizar:

```
.WithName("FinalizarOrden").WithTags("Ordenes")
.WithSummary("Finaliza una orden").Produces<OrdenDto>(200)
.Produces(404).ProducesProblem(409);
```

Por qué funciona así. Los metadatos de una operación declaran rutas, respuestas y esquemas; son documentación de un comportamiento que ya fue implementado y probado.

Verificación concreta. DELETE declara 204/404/409 y finalizar 200/404/409; la documentación coincide con las restricciones de estado del código.

Si algo falla. Si falta un código de respuesta, añade Produces/ProducesProblem al endpoint correspondiente sin inventar comportamientos.

PASO 2.4.16 — Crear SchemaFilter EjemploCrearOrden

Resultado de aprendizaje. Crear SchemaFilter EjemploCrearOrden.

Procedimiento guiado.

1. Vuelve a `DocumentacionExtensions.cs` y añade los `using` de `AceriaApp.Api.Contratos`, `Microsoft.OpenApi.Any` y `Swashbuckle.AspNetCore.SwaggerGen` si no están.
2. **Fuera** de la clase `DocumentacionExtensions`, declara una clase `EjemploCrearOrden : ISchemaFilter` y escribe su `Apply` según el fragmento.
3. Comprueba que solo establece `schema.Example` cuando `context.Type == typeof(CrearOrdenRequest)` y que el objeto de ejemplo contiene cliente, material y cantidad.
4. Dentro del `AddSwaggerGen(options => { ... })` que ya existe, añade `options.SchemaFilter<EjemploCrearOrden>();`. Guarda, compila y vuelve a descargar el JSON de Swagger.

Archivo de trabajo: **`src/AceriaApp.Api/Documentacion/DocumentacionExtensions.cs`**.

```
// Añade al archivo, fuera de la clase DocumentacionExtensions:
public sealed class EjemploCrearOrden : ISchemaFilter
{
    public void Apply(OpenApiSchema schema, SchemaFilterContext context)
    {
        if (context.Type != typeof(CrearOrdenRequest)) return;
        schema.Example = new OpenApiObject
        {
            ["cliente"] = new OpenApiString("ACX-001"),
            ["material"] = new OpenApiString("CHAPA-3MM"),
            ["cantidad"] = new OpenApiInteger(10)
        };
    }
}
```

Registro del filtro dentro de `AddSwaggerGen(options => ...)`:

```
options.SchemaFilter<EjemploCrearOrden>();
```

Por qué funciona así. Un ejemplo JSON ilustra el contrato de entrada. Swagger UI permite usarlo para enviar una petición real, pero no crea datos hasta ejecutar el endpoint.

Verificación concreta. El schema de `CrearOrdenRequest` incorpora un `example` con `ACX-001`, `CHAPA-3MM` y `10`. No se genera ninguna orden hasta ejecutar un POST.

Si algo falla. Si no aparece example, comprueba SchemaFilter y el tipo CrearOrdenRequest usado en el esquema.

PASO 2.4.17 — Inspeccionar example de CrearOrdenRequest en Swagger

Resultado de aprendizaje. Inspeccionar example de CrearOrdenRequest en Swagger.

Procedimiento guiado.

1. En `/swagger/v1/swagger.json` busca el nodo `components.schemas.CrearOrdenRequest`.
2. Expande su propiedad `example` y compara las tres claves con las propiedades del contrato C# `CrearOrdenRequest`.
3. Comprueba que se trata de un ejemplo de **petición**, no de un registro real de `Ordenes`; no esperes encontrar Id, Estado ni FechaCreacion en este cuerpo.
4. Si `example` no aparece, coloca un punto de interrupción en `EjemploCrearOrden.Apply`, comprueba el tipo `context.Type` y la inscripción de `SchemaFilter`.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Por qué funciona así. Un ejemplo JSON ilustra el contrato de entrada. Swagger UI permite usarlo para enviar una petición real, pero no crea datos hasta ejecutar el endpoint.

Verificación concreta. El esquema contiene un ejemplo de JSON de entrada que coincide con `CrearOrdenRequest`; el documento sigue siendo generado automáticamente.

Si algo falla. Si no aparece example, comprueba SchemaFilter y el tipo CrearOrdenRequest usado en el esquema.

PASO 2.4.18 — Ejecutar un POST desde Swagger UI

Resultado de aprendizaje. Ejecutar un POST desde Swagger UI.

Procedimiento guiado.

1. Abre Swagger UI en Development, expande **POST /api/v1/ordenes** y selecciona **Try it out**.

2. Introduce un JSON válido o utiliza el ejemplo documentado con Cliente, Material y Cantidad 10; pulsa **Execute**.
3. Examina la sección de respuesta: **201 Created**, cuerpo `OrdenDto` con Id y cabecera `Location`.
4. Guarda la URL de `Location` para el paso siguiente. Si recibes 400 o 422, revisa los valores enviados y las reglas de validación/capacidad.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Por qué funciona así. Un ejemplo JSON ilustra el contrato de entrada. Swagger UI permite usarlo para enviar una petición real, pero no crea datos hasta ejecutar el endpoint.

Verificación concreta. Swagger UI ha enviado un POST real que creó una orden en LocalDB; se observa 201, el GUID y `Location`.

Si algo falla. Si no aparece example, comprueba SchemaFilter y el tipo CrearOrdenRequest usado en el esquema.

PASO 2.4.19 — Consultar la orden creada desde la propia UI

Resultado de aprendizaje. Consultar la orden creada desde la propia UI.

Procedimiento guiado.

1. Sin salir de Swagger UI, expande **GET /api/v1/ordenes/{id}** y pulsa **Try it out**.
2. Copia únicamente el GUID de la respuesta creada en 2.4.18, sin incluir la ruta completa, en el parámetro de la operación GET.
3. Pulsa **Execute** y verifica **200**. Compara Cliente, Material, Cantidad y Estado con el DTO devuelto en el POST.
4. Ejecuta otro GET con un GUID válido pero inexistente para comprobar que Swagger permite observar también la respuesta 404.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Por qué funciona así. Un ejemplo JSON ilustra el contrato de entrada. Swagger UI permite usarlo para enviar una petición real, pero no crea datos hasta ejecutar el endpoint.

Verificación concreta. GET por id devuelve los datos persistidos por POST; la documentación no sustituye al servidor ni a sus comprobaciones.

Si algo falla. Si no aparece example, comprueba SchemaFilter y el tipo CrearOrdenRequest usado en el esquema.

PASO 2.4.20 — Crear tests HTTP del documento OpenAPI real

Resultado de aprendizaje. Crear tests HTTP del documento OpenAPI real.

Procedimiento guiado.

1. En el proyecto de tests heredado crea `ContratoOpenApiTests.cs` con namespace `AceriaApp.Api.Tests`.
2. Copia **el archivo completo inicial** del fragmento 2.4.20, incluidos los `using System.Net`, `System.Text.Json`, `Microsoft.AspNetCore.Hosting` y `Xunit`.
3. En `OpenApi_ContieneRutasYVersion` sigue la solicitud a `/swagger/v1/swagger.json`, el 200, la lectura de `info.version` y la comprobación de las rutas GET/POST.
4. Guarda y ejecuta el test. Los pasos 2.4.22 y 2.4.23 añadirán métodos dentro de **esta misma clase**, no otra clase con el mismo nombre.

Archivo de trabajo: `tests/AceriaApp.Api.Tests/ContratoOpenApiTests.cs`.

```
using System.Net;
using System.Text.Json;
using Microsoft.AspNetCore.Hosting;
using Xunit;

namespace AceriaApp.Api.Tests;

public class ContratoOpenApiTests
{
    [Fact]
    public async Task OpenApi_ContieneRutasYVersion()
    {
        using var f = new ApiFactory();
        var r = await f.CreateClient().GetAsync("/swagger/v1/swagger.json");
        Assert.Equal(HttpStatusCode.OK, r.StatusCode);
        using var json = JsonDocument.Parse(await r.Content.ReadAsStringAsync());
        Assert.Equal("v1", json.RootElement.GetProperty("info")
            .GetProperty("version").GetString());
        var paths = json.RootElement.GetProperty("paths");
        Assert.True(paths.TryGetProperty("/api/v1/ordenes", out var resource));
        Assert.True(resource.TryGetProperty("get", out _));
        Assert.True(resource.TryGetProperty("post", out _));
    }
}
```

Por qué funciona así. Descargar el JSON OpenAPI con WebApplicationFactory y comprobar info, paths y schemas es una prueba contractual real; en Production debe respetarse el 404.

Verificación concreta. `ContratoOpenApiTests` ejecuta una petición HTTP real y comprueba la versión y las rutas; sigue disponible la suite de validación heredada.

Si algo falla. Si el JSON no incluye rutas, comprueba MapGet/MapPost y que las pruebas apuntan al servidor correcto.

PASO 2.4.21 — Comprobar rutas GET y POST en paths

Resultado de aprendizaje. Comprobar rutas GET y POST en paths.

Procedimiento guiado.

1. Reabre el test `OpenApi_ContieneRutasYVersion` de 2.4.20 y localiza la lectura de `GetProperty("info")` y `GetProperty("paths")`.
2. Coloca un punto de interrupción tras analizar el documento y examina que `info.version` es `v1` y que `/api/v1/ordenes` tiene los métodos `get` y `post`.
3. Ejecuta `dotnet test` filtrando por el nombre del método o usa **Ejecutar prueba** sobre este caso en el Explorador de pruebas.
4. No copies otra clase ni un nuevo documento JSON para esta actividad: solo interpretas la evidencia del test ya creado.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Por qué funciona así. Descargar el JSON OpenAPI con WebApplicationFactory y comprobar info, paths y schemas es una prueba contractual real; en Production debe respetarse el 404.

Verificación concreta. El test de contrato se mantiene verde y accede a operaciones reales declaradas por los handlers de la API.

Si algo falla. Si el JSON no incluye rutas, comprueba MapGet/MapPost y que las pruebas apuntan al servidor correcto.

PASO 2.4.22 — Comprobar schema y ejemplo de petición

Resultado de aprendizaje. Comprobar schema y ejemplo de petición.

Procedimiento guiado.

1. Abre `ContratoOpenApiTests.cs` y sitúa el cursor dentro de su única clase pública, antes de la llave final.
2. Agrega el método `OpenApi_IncluyeEjemplo()` del bloque, conservando el test de rutas y versión.
3. Comprueba que consulta el JSON real, accede a `components.schemas`, encuentra `CrearOrdenRequest` y exige que exista `example`.
4. Ejecuta el nuevo test y vuelve a revisar el esquema si falla: el ejemplo depende de que `SchemaFilter<EjemploCrearOrden>` se registre en la etapa 2.4.16.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Código que se incorpora en este paso:

```
[Fact]
public async Task OpenApi_IncluyeEjemplo()
{
    using var f = new ApiFactory();
    var r = await f.CreateClient().GetAsync("/swagger/v1/swagger.json");
    using var json = JsonDocument.Parse(await r.Content.ReadAsStringAsync());
    var schemas = json.RootElement
        .GetProperty("components").GetProperty("schemas");
    Assert.True(schemas.TryGetProperty("CrearOrdenRequest", out var schema));
    Assert.True(schema.TryGetProperty("example", out _));
}
```

Por qué funciona así. Descargar el JSON OpenAPI con WebApplicationFactory y comprobar info, paths y schemas es una prueba contractual real; en Production debe respetarse el 404.

Verificación concreta. El test del ejemplo pasa y comprueba el schema producido por el servidor, no la presencia de una cadena escrita manualmente.

Si algo falla. Si el JSON no incluye rutas, comprueba MapGet/MapPost y que las pruebas apuntan al servidor correcto.

PASO 2.4.23 — Comprobar que Production responde 404 para Swagger

Resultado de aprendizaje. Comprobar que Production responde 404 para Swagger.

Procedimiento guiado.

1. Dentro de esa misma clase agrega `Produccion_NoExponeSwagger()` según el fragmento del paso.
2. Observa `f.WithWebHostBuilder(b => b.UseEnvironment("Production"))`: se crea otro host de prueba con un entorno diferente al de Development.
3. Ejecuta GET `/swagger/v1/swagger.json` en ese host y comprueba **404**. Este test no debe borrar ni alterar los endpoints REST.
4. Ejecuta la suite acumulada de la etapa 2.4 y comprueba que los tres tests de contrato funcionan junto con los heredados.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Código que se incorpora en este paso:

```
[Fact]
public async Task Produccion_NoExponeSwagger()
```

```
{
    using var f = new ApiFactory();
    using var other = f.WithWebHostBuilder(
        b => b.UseEnvironment("Production"));
    var r = await other.CreateClient()
        .GetAsync("/swagger/v1/swagger.json");
    Assert.Equal(HttpStatusCode.NotFound, r.StatusCode);
}
```

Por qué funciona así. Descargar el JSON OpenAPI con WebApplicationFactory y comprobar info, paths y schemas es una prueba contractual real; en Production debe respetarse el 404.

Verificación concreta. En Production Swagger no se publica (404); en Development el documento sigue devolviendo 200. La suite base de 2.4 alcanza 15 tests sin fallos.

Si algo falla. Si el JSON no incluye rutas, comprueba MapGet/MapPost y que las pruebas apuntan al servidor correcto.

PASO 2.4.24 — Reto: documentar PATCH y verificar su summary en JSON

Resultado de aprendizaje. Reto: documentar PATCH y verificar su summary en JSON.

Procedimiento guiado.

1. Reto opcional: trabaja en una copia separada del punto 2.4; no cambies el proyecto de referencia que utilizará el módulo siguiente.
2. En `Program.cs`, localiza el PATCH de cantidad y modifica solo el texto de su `.WithSummary(...)`, conservando handler, filtros y códigos de respuesta.
3. Reinicia el servidor y solicita otra vez `/swagger/v1/swagger.json`; busca en `paths` la operación `patch` correspondiente a la ruta `/cantidad`.
4. Comprueba que su propiedad `summary` contiene el nuevo texto. Si la UI conserva una versión anterior, actualiza la página o vuelve a descargar el JSON generado.

Por qué funciona así. Cambiar un WithSummary y ver reflejada la modificación en el JSON demuestra la vinculación de la documentación con el código ejecutado.

Verificación concreta. El reto demuestra que OpenAPI refleja los metadatos ejecutables tras reiniciar; no requiere editar un archivo Swagger estático.

Si algo falla. Si el summary no cambia, modifica WithSummary en el endpoint real y vuelve a solicitar el documento.

Caso práctico completo — Contrato de ejemplo y prueba en Production

La sección `components.schemas` documenta objetos que pueden reutilizarse desde varias operaciones. Busca `CrearOrdenRequest`. Debe exponer las propiedades cliente, material y cantidad. El `SchemaFilter` `EjemploCrearOrden` incorpora valores orientativos para esos campos; no crea ninguna orden. El propósito es ayudar a quien consume el API a preparar un cuerpo JSON compatible.

Desde Swagger UI abre el POST de órdenes y utiliza Try it out. Envía un JSON con ACX-001, CHAPA-3MM y cantidad 10; el servidor debe devolver 201 y Location. Guarda el identificador y solicita GET de detalle desde la misma interfaz. Si obtienes una respuesta distinta, revisa el cuerpo, las reglas de validación y la conexión a la base de datos; no concluyas que el contrato está bien porque Swagger haya dibujado el formulario.

Para probar la documentación automáticamente, abre `ContratoOpenApiTests.cs`. La prueba arranca WebApplicationFactory, solicita el JSON real y analiza sus propiedades con JsonDocument. Comprueba `info.version`, las rutas de `paths` y el esquema de CrearOrdenRequest con su `example`. La prueba debe fallar si el endpoint POST se elimina sin actualizar el contrato de la API.

Finalmente crea la fábrica de tests con `UseEnvironment("Production")` y vuelve a solicitar `/swagger/v1/swagger.json`. El método `UseDocumentacion` restringe la publicación a Development; por eso la respuesta esperada es 404. Esta comprobación enseña a separar la presencia de un paquete de su exposición pública.

Reto final: modifica `WithSummary` de PATCH, arranca de nuevo y comprueba en el JSON de `paths` que el resumen se actualizó. No escribas un documento OpenAPI manual: la prueba debe demostrar que la documentación sigue derivándose de la implementación.

Matriz de comprobación del punto

Comportamiento	Resultado esperado
`GET /swagger/v1/swagger.json`	200 en Development
`info.version`	v1
`paths contiene GET y POST`	Rutas descubiertas
`schemas contiene CrearOrdenRequest`	Contrato registrado
`GET /swagger/v1/swagger.json`	404 en Production

Reto integrador. Reconstruye sin consultar la solución el último endpoint o validación practicada y añade una prueba HTTP que demuestre su resultado. Describe el recurso, el verbo, el contrato y el código devuelto. Si no puedes reproducir una respuesta, vuelve al primer paso que introdujo el comportamiento.
